

1.1 Introduction

Internet of Things (IoT) comprises things that have unique identities and are connected to the Internet. While many existing devices, such as networked computers or 4G-enabled mobile phones, already have some form of unique identities and are also connected to the Internet, the focus on IoT is in the configuration, control and networking via the Internet of devices or "things" that are traditionally not associated with the Internet. These include devices such as thermostats, utility meters, a bluetooth-connected headset, irrigation pumps and sensors, or control circuits for an electric car's engine. Internet of Things is a new revolution in the capabilities of the endpoints that are connected to the Internet, and is being driven by the advancements in capabilities (in combination with lower costs) in sensor networks, mobile devices, wireless communications, networking and cloud technologies. Experts forecast that by the year 2020 there will be a total of 50 billion devices/things connected to the Internet. Therefore, the major industry players are excited by the prospects of new markets for their products. The products include hardware and software components for IoT endpoints, hubs, or control centers of the IoT universe.

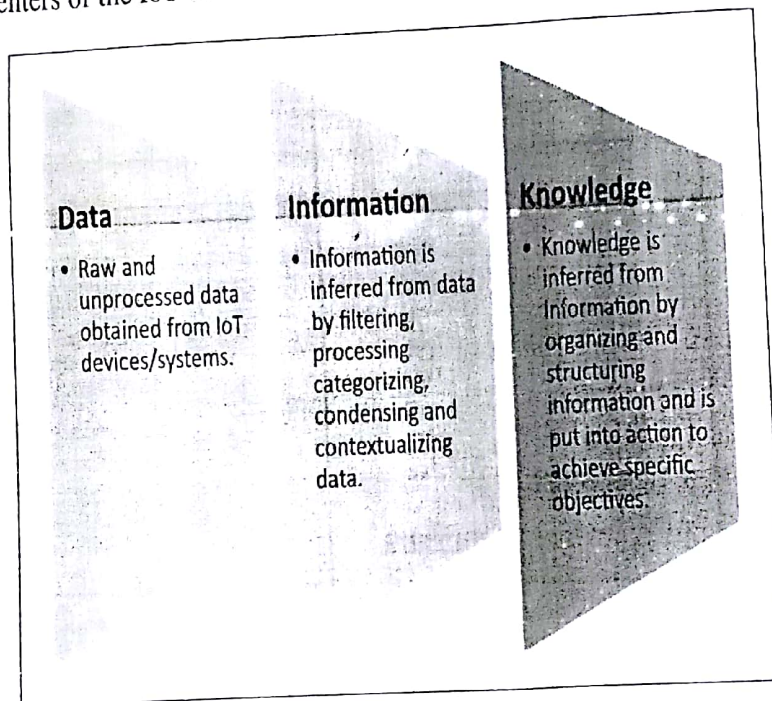


Figure 1.1: Inferring information and knowledge from data

The scope of IoT is not limited to just connecting things (devices, appliances, machines) to the Internet. IoT allows these things to communicate and exchange data (control & information, that could include data associated with users) while executing meaningful applications towards a common user or machine goal. Data itself does not have a meaning

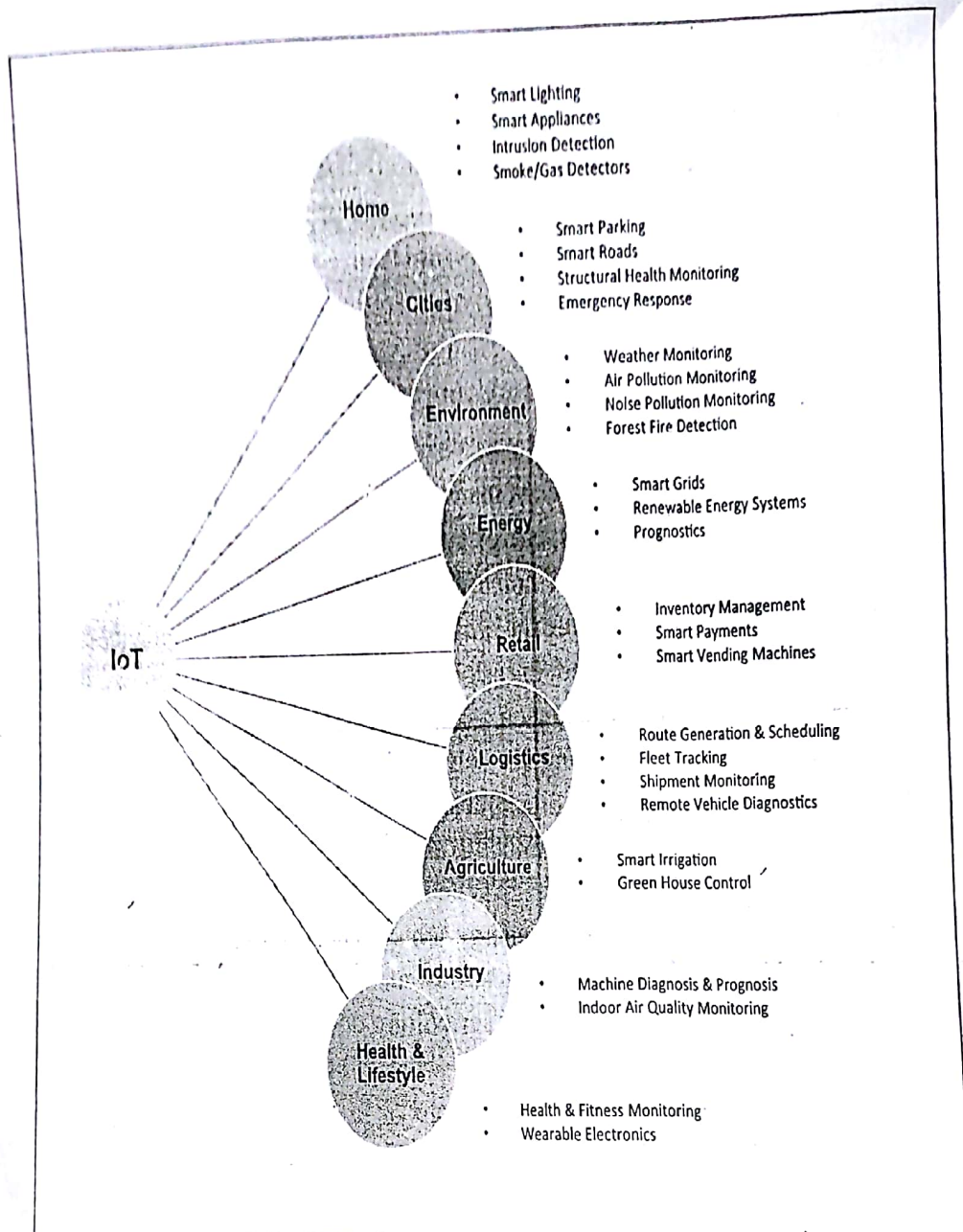


Figure 1.2: Applications of IoT

until it is contextualized processed into useful information. Applications on IoT networks extract and create information from lower level data by filtering, processing, categorizing, condensing and contextualizing the data. This information obtained is then organized and structured to infer knowledge about the system and/or its users, its environment, and its operations and progress towards its objectives, allowing a smarter performance, as shown in Figure 1.1. For example, consider a series of raw sensor measurements $((72,45); (84, 56))$ generated by a weather monitoring station, which by themselves do not have any meaning or

context. To give meaning to the data, a context is added. Each tuple in data represents the temperature and humidity. With the context added we know the meaning (or information) of the data. The information is obtained by categorizing, condensing, and summarizing the average temperature and humidity readings for the last five data tuples. The next step is to organize the data into meaningful relationships between pieces of information to infer knowledge. For example, an alert is raised if the average temperature exceeds 120°F and this alert may be conditioned on the user's geographical position as well.

The applications of Internet of Things span a wide range of domains including (but not limited to) homes, cities, environment, energy systems, transportation, logistics, industry, agriculture, and health as listed in Figure 1.2. For homes, IoT has applications such as smart lighting that adapt the lighting to suit the ambient conditions, smart appliances that can be remotely monitored and controlled, intrusion detection systems, smart smoke detectors, etc. For cities, IoT has applications such as smart parking systems that provide status updates on available slots, smart lighting that helps in saving energy, smart roads that provide information on driving conditions and structural health monitoring systems. For environment, IoT has applications such as weather monitoring, air and water quality monitoring, forest fire detection systems, etc. For energy systems, IoT has applications such as smart grids, grid integration of renewable energy sources, etc. For retail domain, IoT has applications such as smart inventory management, smart payments and smart vending machines. For agriculture, IoT has applications such as smart irrigation systems that help in saving water, smart green house control systems. Industrial applications of IoT include machine diagnosis and prognosis systems that help in predicting faults and determining the cause of faults and indoor air quality systems. For health and lifestyle, IoT has applications such as health and fitness monitoring systems and wearable electronics.

high in this example can be that measured every minute. With this we measured data tuples. Further processing this data. For example, the average temperature for the last five minutes is obtained by averaging the information and understand the average which can be put into action. The last five minutes exceeds 120°F, and position as well.

range of domains including (but not limited to) homes, cities, environment, energy systems, transportation, logistics, industry, agriculture, and health as listed in Figure 1.2. For homes, IoT has applications such as smart lighting that adapt the lighting to suit the ambient conditions, smart appliances that can be remotely monitored and controlled, intrusion detection systems, smart smoke detectors, etc. For cities, IoT has applications such as smart parking systems that provide status updates on available slots, smart lighting that helps in saving energy, smart roads that provide information on driving conditions and structural health monitoring systems. For environment, IoT has applications such as weather monitoring, air and water quality monitoring, forest fire detection systems, etc. For energy systems, IoT has applications such as smart grids, grid integration of renewable energy sources, etc. For retail domain, IoT has applications such as smart inventory management, smart payments and smart vending machines. For agriculture, IoT has applications such as smart irrigation systems that help in saving water, smart green house control systems. Industrial applications of IoT include machine diagnosis and prognosis systems that help in predicting faults and determining the cause of faults and indoor air quality systems. For health and lifestyle, IoT has applications such as health and fitness monitoring systems and wearable electronics.

1.1.1 Definition & Characteristics of IoT

The Internet of Things (IoT) has been defined as [1]:

Definition: A dynamic global network infrastructure based on standard and interoperable communication protocols where physical and virtual "things" have identities, physical attributes, and virtual interfaces, and are seamlessly integrated into the information network, often communicate data associated with users and their environments.

self-configuring capabilities, protocols where physical and virtual personalities and use intelligent information network, often communicate data associated with users and their environments.

Let us examine this definition of IoT further to put some of the terms into perspective.

- **Dynamic & Self-Adapting:** IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating conditions, user's context, or sensed environment. For example, consider a surveillance system comprising of a number of surveillance cameras. The surveillance cameras can adapt their modes (to normal or infra-red night modes) based on whether it is day or night. Cameras could switch from lower resolution to higher resolution modes when any motion is detected and alert nearby cameras to do the same. In this example, the surveillance system is adapting itself based on the context and changing (e.g., dynamic) conditions.
- **Self-Configuring:** IoT devices may have self-configuring capability, allowing a large number of devices to work together to provide certain functionality (such as weather monitoring). These devices have the ability to configure themselves (in association with the IoT infrastructure), setup the networking, and fetch latest software upgrades with minimal manual or user intervention.
- **Interoperable Communication Protocols:** IoT devices may support a number of interoperable communication protocols and can communicate with other devices and also with the infrastructure. We describe some of the commonly used communication protocols and models in later sections.
- **Unique Identity:** Each IoT device has a unique identity and a unique identifier (such as an IP address or a URI). IoT systems may have intelligent interfaces which adapt based on the context, allow communicating with users and the environmental contexts. IoT device interfaces allow users to query the devices, monitor their status, and control them remotely, in association with the control, configuration and management infrastructure.
- **Integrated into Information Network:** IoT devices are usually integrated into the information network that allows them to communicate and exchange data with other devices and systems. IoT devices can be dynamically discovered in the network, by other devices and/or the network, and have the capability to describe themselves (and their characteristics) to other devices or user applications. For example, a weather monitoring node can describe its monitoring capabilities to another connected node so that they can communicate and exchange data. Integration into the information network helps in making IoT systems "smarter" due to the collective intelligence of the individual devices in collaboration with the infrastructure. Thus, the data from a large number of connected weather monitoring IoT nodes can be aggregated and analyzed to predict the weather.

1.2 Physical Design of IoT

1.2.1 Things in IoT

The "Things" in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, actuating and monitoring capabilities. IoT devices can exchange data with other connected devices and applications (directly or indirectly), or collect data from other devices and process the data either locally or send the data to centralized servers or cloud-based application back-ends for processing the data, or perform some tasks locally and other tasks within the IoT infrastructure, based on temporal and space constraints (i.e., memory, processing capabilities, communication latencies and speeds, and deadlines).

Figure 1.3 shows a block diagram of a typical IoT device. An IoT device may consist of several interfaces for connections to other devices, both wired and wireless. These include (i) I/O interfaces for sensors, (ii) interfaces for Internet connectivity, (iii) memory and storage interfaces and (iv) audio/video interfaces. An IoT device can collect various types of data from the on-board or attached sensors, such as temperature, humidity, light intensity. The sensed data can be communicated either to other devices or cloud-based servers/storage. IoT devices can be connected to actuators that allow them to interact with other physical entities (including non-IoT devices and systems) in the vicinity of the device. For example, a relay switch connected to an IoT device can turn an appliance on/off based on the commands sent to the IoT device over the Internet.)

IoT devices can also be of varied types, for instance, wearable sensors, smart watches, LED lights, automobiles and industrial machines. Almost all IoT devices generate data in some form or the other which when processed by data analytics systems leads to useful information to guide further actions locally or remotely. For instance, sensor data generated by a soil moisture monitoring device in a garden, when processed can help in determining the optimum watering schedules. Figure 1.4 shows different types of IoT devices.

1.2.2 IoT Protocols

Link Layer

Link layer protocols determine how the data is physically sent over the network's physical layer or medium (e.g., copper wire, coaxial cable, or a radio wave). The scope of the link layer is the local network connection to which host is attached. Hosts on the same link exchange data packets over the link layer using link layer protocols. Link layer determines how the packets are coded and signaled by the hardware device over the medium to which the host is attached (such as a coaxial cable). Let us now look at some link layer protocols which are relevant in the context of IoT.

- **802.3 - Ethernet** : IEEE 802.3 is a collection of wired Ethernet standards for the link

1.2 Physical Design of IoT

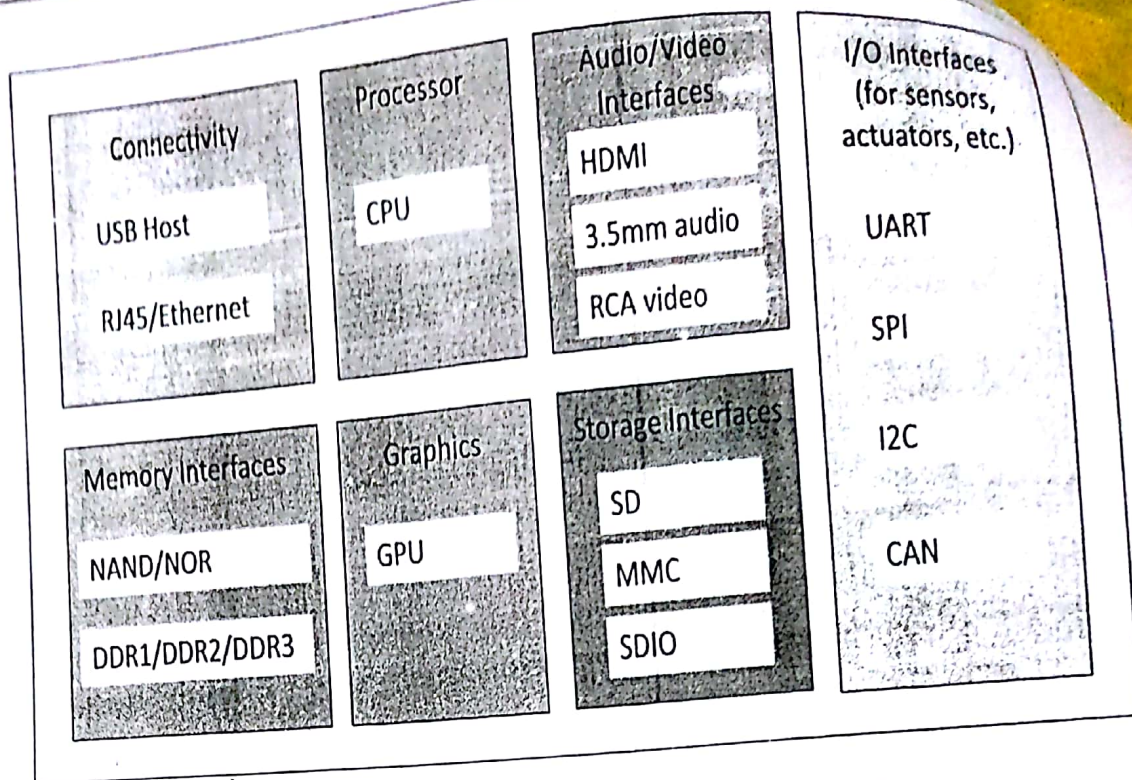


Figure 1.3: Generic block diagram of an IoT Device

layer. For example, 802.3 is the standard for 10BASE5 Ethernet that uses coaxial cable as a shared medium, 802.3.i is the standard for 10BASE-T Ethernet over copper twisted-pair connections, 802.3.j is the standard for 10BASE-F Ethernet over fiber optic connections, 802.3ae is the standard for 10 Gbit/s Ethernet over fiber, and so on. These standards provide data rates from 10 Mb/s to 40 Gb/s and higher. The shared medium in Ethernet can be a coaxial cable, twisted-pair wire or an optical fiber. The shared medium (i.e., broadcast medium) carries the communication for all the devices on the network, thus data sent by one device can be received by all devices subject to propagation conditions and transceiver capabilities. The specifications of the 802.3 standards are available on the IEEE 802.3 working group website [2].

- **802.11 - WiFi :** IEEE 802.11 is a collection of wireless local area network (WLAN) communication standards, including extensive description of the link layer. For example, 802.11a operates in the 5 GHz band, 802.11b and 802.11g operate in the 2.4 GHz band, 802.11n operates in the 2.4/5 GHz bands, 802.11ac operates in the 5 GHz band and 802.11ad operates in the 60 GHz band. These standards provide data rates from 1 Mb/s to upto 6.75 Gb/s. The specifications of the 802.11 standards are available on the IEEE 802.11 working group website [3]
- **802.16 - WiMax :** IEEE 802.16 is a collection of wireless broadband standards, including extensive descriptions for the link layer (also called WiMax). WiMax

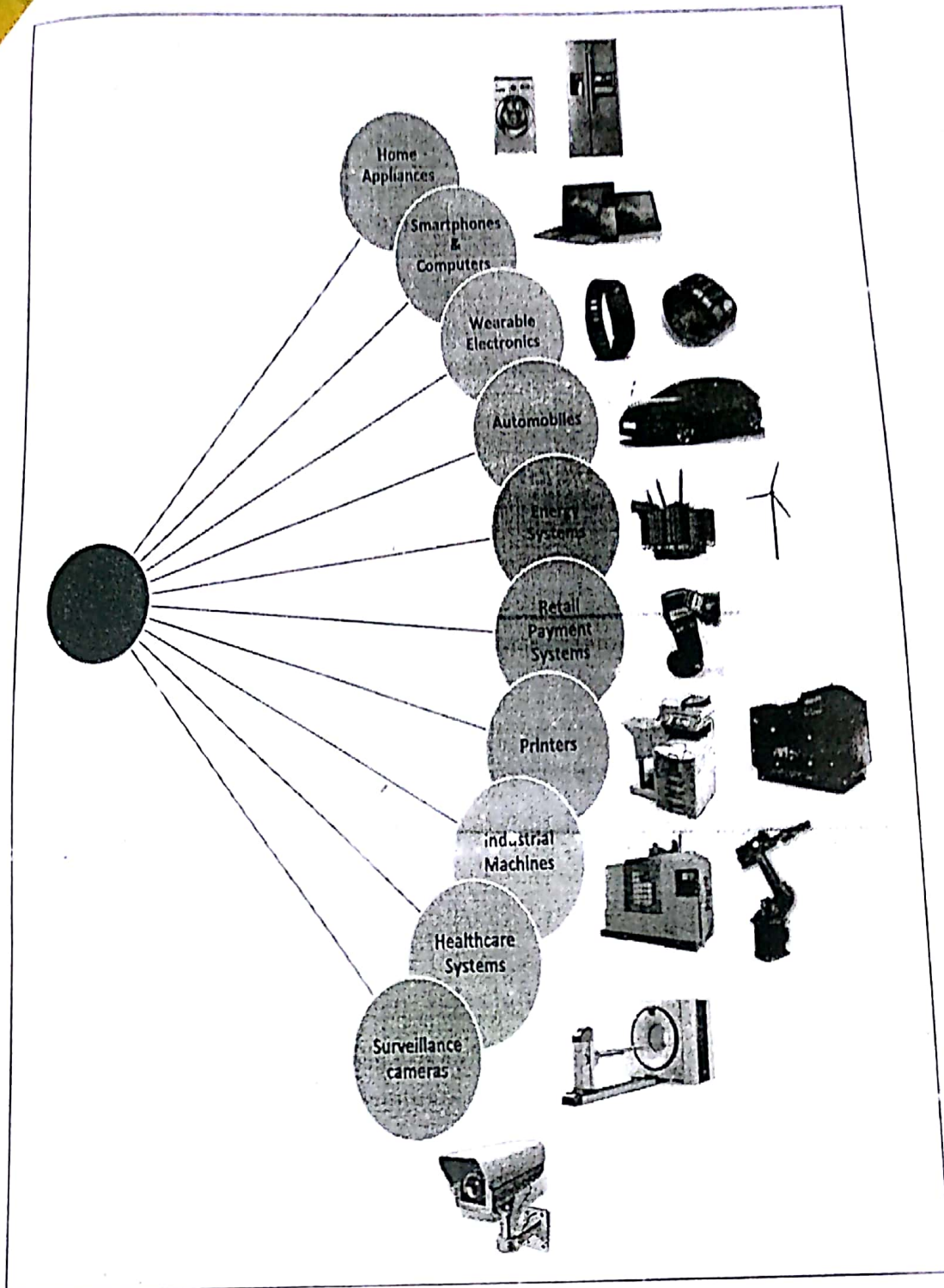


Figure 1.4: IoT Devices

standards provide data rates from 1.5 Mb/s to 1 Gb/s. The recent update (802.16m) provides data rates of 100 Mbit/s for mobile stations and 1 Gbit/s for fixed stations. The specifications of the 802.11 standards are readily available on the IEEE 802.16 working group website [4]

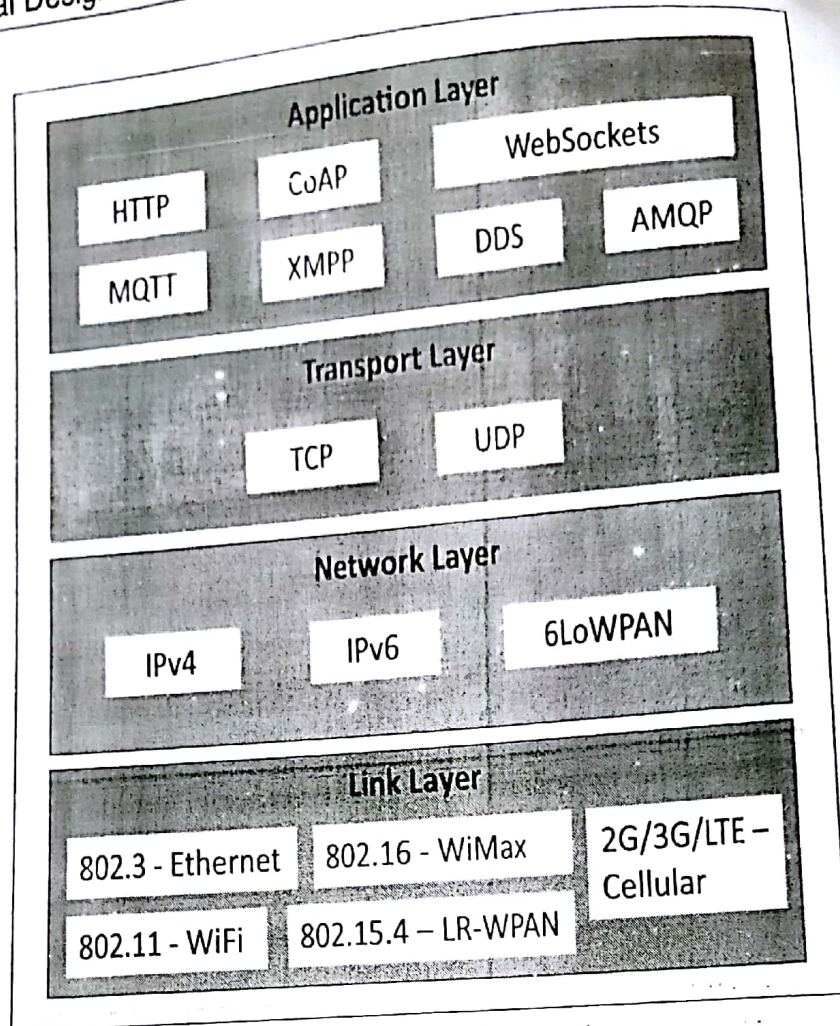


Figure 1.5: IoT Protocols

- **802.15.4 - LR-WPAN** : IEEE 802.15.4 is a collection of standards for low-rate wireless personal area networks (LR-WPANs). These standards form the basis of specifications for high level communication protocols such as ZigBee. LR-WPAN standards provide data rates from 40 Kb/s 250 Kb/s. These standards provide low-cost and low-speed communication for power constrained devices. The specifications of the 802.15.4 standards are available on the IEEE 802.15 working group website [5]
- **2G/ 3G/ 4G - Mobile Communication** : There are different generations of mobile communication standards including second generation (2G including GSM and CDMA), third generation (3G - including UMTS and CDMA2000) and fourth generation (4G - including LTE). IoT devices based on these standards can communicate over cellular networks. Data rates for these standards range from 9.6 Kb/s (for 2G) to upto 100 Mb/s (for 4G) and are available from the 3GPP websites.

Network/Internet Layer

The network layers are responsible for sending of IP datagrams from the source network to the destination network. This layer performs the host addressing and packet routing. The datagrams contain the source and destination addresses which are used to route them from the source to destination across multiple networks. Host identification is done using hierarchical IP addressing schemes such as IPv4 or IPv6.

- **IPv4** : Internet Protocol version 4 (IPv4) is the most deployed Internet protocol that is used to identify the devices on a network using a hierarchical addressing scheme. IPv4 uses a 32-bit address scheme that allows total of 2^{32} or 4,294,967,296 addresses. As more and more devices got connected to the Internet, these addresses got exhausted in the year 2011. IPv4 has been succeeded by IPv6. The IP protocols establish connections on packet networks, but do not guarantee delivery of packets. Guaranteed delivery and data integrity are handled by the upper layer protocols (such as TCP). IPv4 is formally described in RFC 791 [6].
- **IPv6** : Internet Protocol version 6 (IPv6) is the newest version of Internet protocol and successor to IPv4. IPv6 uses 128-bit address scheme that allows total of 2^{128} or 3.4×10^{38} addresses. IPv6 is formally described in RFC 2460 [7].
- **6LoWPAN** : 6LoWPAN (IPv6 over Low power Wireless Personal Area Networks) brings IP protocol to the low-power devices which have limited processing capability. 6LoWPAN operates in the 2.4 GHz frequency range and provides data transfer rates of 250 Kb/s. 6LoWPAN works with the 802.15.4 link layer protocol and defines compression mechanisms for IPv6 datagrams over IEEE 802.15.4-based networks [8].

Transport Layer

The transport layer protocols provide end-to-end message transfer capability independent of the underlying network. The message transfer capability can be set up on connections, either using handshakes (as in TCP) or without handshakes/acknowledgements (as in UDP). The transport layer provides functions such as error control, segmentation, flow control and congestion control.

- **TCP** : Transmission Control Protocol (TCP) is the most widely used transport layer protocol, that is used by web browsers (along with HTTP, HTTPS application layer protocols), email programs (SMTP application layer protocol) and file transfer (FTP). TCP is a connection oriented and stateful protocol. While IP protocol deals with sending packets, TCP ensures reliable transmission of packets in-order. TCP also provides error detection capability so that duplicate packets can be discarded and lost packets are retransmitted. The flow control capability of TCP ensures that

rate at which the sender sends the data is not too high for the receiver to process. The congestion control capability of TCP helps in avoiding network congestion and congestion collapse which can lead to degradation of network performance. TCP is described in RFC 793 [9].

- **UDP** : Unlike TCP, which requires carrying out an initial setup procedure, UDP is a connectionless protocol. UDP is useful for time-sensitive applications that have very small data units to exchange and do not want the overhead of connection setup. UDP is a transaction oriented and stateless protocol. UDP does not provide guaranteed delivery, ordering of messages and duplicate elimination. Higher levels of protocols can ensure reliable delivery or ensuring connections created are reliable. UDP is described in RFC 768 [10].

Application Layer

Application layer protocols define how the applications interface with the lower layer protocols to send the data over the network. The application data, typically in files, is encoded by the application layer protocol and encapsulated in the transport layer protocol which provides connection or transaction oriented communication over the network. Port numbers are used for application addressing (for example port 80 for HTTP, port 22 for SSH, etc.). Application layer protocols enable process-to-process connections using ports.

- **HTTP** : Hypertext Transfer Protocol (HTTP) is the application layer protocol that forms the foundation of the World Wide Web (WWW). HTTP includes commands such as GET, PUT, POST, DELETE, HEAD, TRACE, OPTIONS, etc. The protocol follows a request-response model where a client sends requests to a server using the HTTP commands. HTTP is a stateless protocol and each HTTP request is independent of the other requests. An HTTP client can be a browser or an application running on the client (e.g., an application running on an IoT device, a mobile application or other software). HTTP protocol uses Universal Resource Identifiers (URIs) to identify HTTP resources. HTTP is described in RFC 2616 [11].
- **CoAP** : Constrained Application Protocol (CoAP) is an application layer protocol for machine-to-machine (M2M) applications, meant for constrained environments with constrained devices and constrained networks. Like HTTP, CoAP is a web transfer protocol and uses a request-response model, however it runs on top of UDP instead of TCP. CoAP uses a client-server architecture where clients communicate with servers using connectionless datagrams. CoAP is designed to easily interface with HTTP. Like HTTP, CoAP supports methods such as GET, PUT, POST, and DELETE. CoAP draft specifications are available on IETF Constrained environments (CoRE) Working Group website [12].

- **WebSocket** : WebSocket protocol allows full-duplex communication over a single socket connection for sending messages between client and server. WebSocket is based on TCP and allows streams of messages to be sent back and forth between the client and server while keeping the TCP connection open. The client can be a browser, a mobile application or an IoT device. WebSocket is described in RFC 6455 [13].
- **MQTT** : Message Queue Telemetry Transport (MQTT) is a light-weight messaging protocol based on the publish-subscribe model. MQTT uses a client-server architecture where the client (such as an IoT device) connects to the server (also called MQTT Broker) and publishes messages to topics on the server. The broker forwards the messages to the clients subscribed to topics. MQTT is well suited for constrained environments where the devices have limited processing and memory resources and the network bandwidth is low. MQTT specifications are available on IBM developerWorks [14].
- **XMPP** : Extensible Messaging and Presence Protocol (XMPP) is a protocol for real-time communication and streaming XML data between network entities. XMPP powers wide range of applications including messaging, presence, data syndication, gaming, multi-party chat and voice/video calls. XMPP allows sending small chunks of XML data from one network entity to another in near real-time. XMPP is a decentralized protocol and uses a client-server architecture. XMPP supports both client-to-server and server-to-server communication paths. In the context of IoT, XMPP allows real-time communication between IoT devices. XMPP is described in RFC 6120 [15].
- **DDS** : Data Distribution Service (DDS) is a data-centric middleware standard for device-to-device or machine-to-machine communication. DDS uses a publish-subscribe model where publishers (e.g. devices that generate data) create topics to which subscribers (e.g., devices that want to consume data) can subscribe. Publisher is an object responsible for data distribution and the subscriber is responsible for receiving published data. DDS provides quality-of-service (QoS) control and configurable reliability. DDS is described in Object Management Group (OMG) DDS specification [16].
- **AMQP** : Advanced Message Queuing Protocol (AMQP) is an open application layer protocol for business messaging. AMQP supports both point-to-point and publisher/subscriber models, routing and queuing. AMQP brokers receive messages from publishers (e.g., devices or applications that generate data) and route them over connections to consumers (applications that process data). Publishers publish the messages to exchanges which then distribute message copies to queues. Messages are either delivered by the broker to the consumers which have subscribed to the queues or the consumers can pull the messages from the queues. AMQP specification is

available on the AMQP working group website [17].

1.3 Logical Design of IoT

Logical design of an IoT system refers to an abstract representation of the entities and processes without going into the low-level specifics of the implementation. In this section we describe the functional blocks of an IoT system and the communication APIs that are used for the examples in this book. The steps in logical design are described in additional detail in Chapter-5.

1.3.1 IoT Functional Blocks

An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication, and management as shown in Figure 1.6. These functional blocks are described as follows:

- **Device** : An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions. You learned about IoT devices in section 1.2.
- **Communication** : The communication block handles the communication for the IoT system. You learned about various protocols used for communication by IoT systems in section 1.2.
- **Services** : An IoT system uses various types of IoT services such as services for device monitoring, device control services, data publishing services and services for device discovery.
- **Management** : Management functional block provides various functions to govern the IoT system.
- **Security** : Security functional block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- **Application** : IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view or analyze the processed data.

1.3.2 IoT Communication Models

- **Request-Response** : Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client. Request-Response model is a stateless communication model and each request-response pair is independent of others. Figure 1.7 shows the client-server

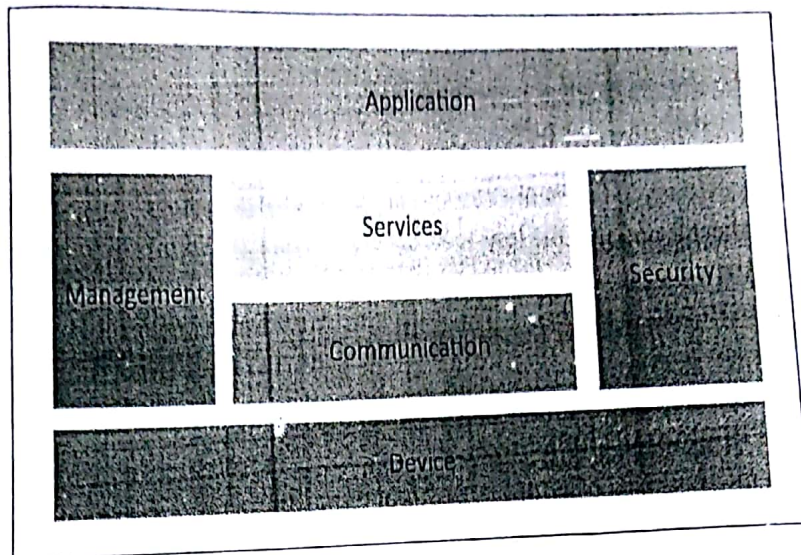


Figure 1.6: Functional Blocks of IoT

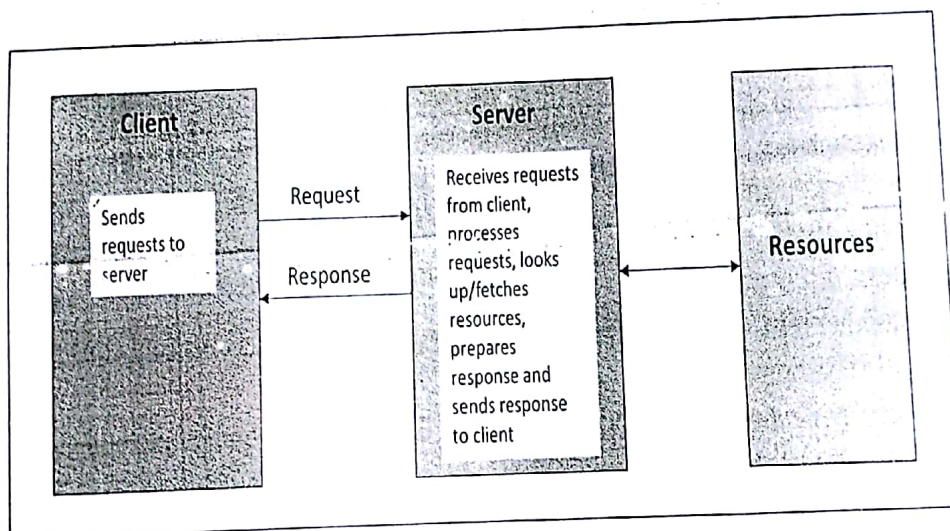


Figure 1.7: Request-Response communication model

- interactions in the request-response model.
- Publish-Subscribe** : Publish-Subscribe is a communication model that involves publishers, brokers and consumers. Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers. Consumers subscribe to the topics which are managed by the broker. When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers. Figure 1.8 shows the publisher-broker-consumer interactions

1.3 Logical Design of IoT

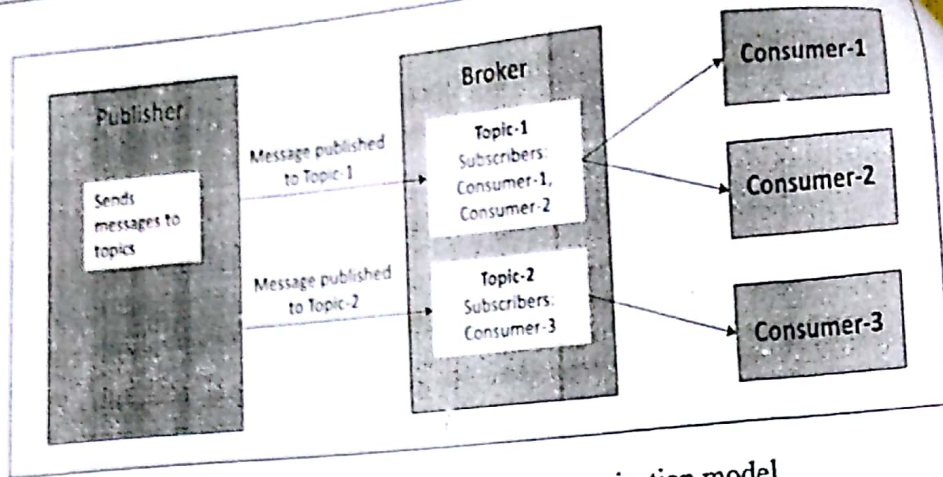


Figure 1.8: Publish-Subscribe communication model

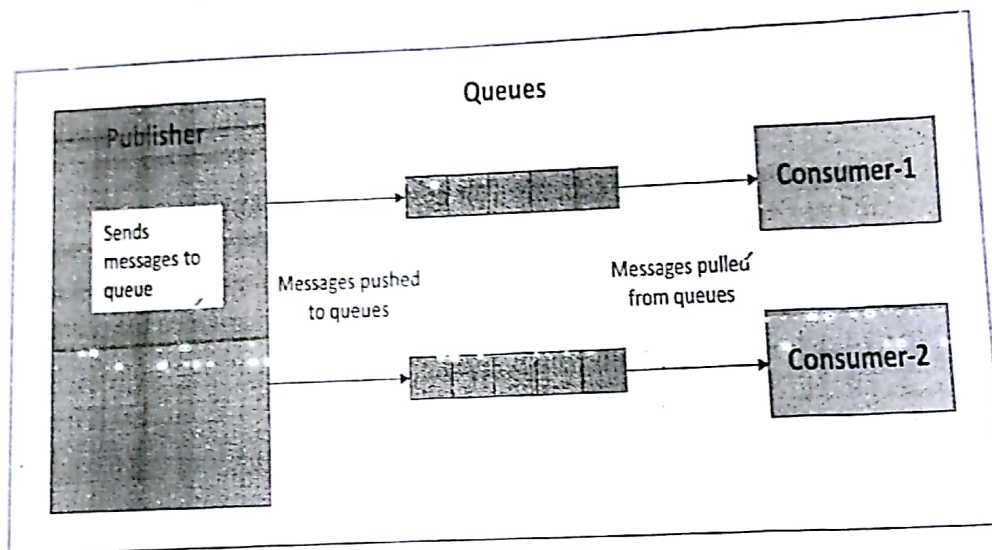


Figure 1.9: Push-Pull communication model

in the publish-subscribe model.

- **Push-Pull:** Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers. Queues help in decoupling the messaging between the producers and consumers. Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull data. Figure 1.9 shows the publisher-queue-consumer interactions in the push-pull model.
- **Exclusive Pair:** Exclusive Pair is a bi-directional, fully duplex communication model

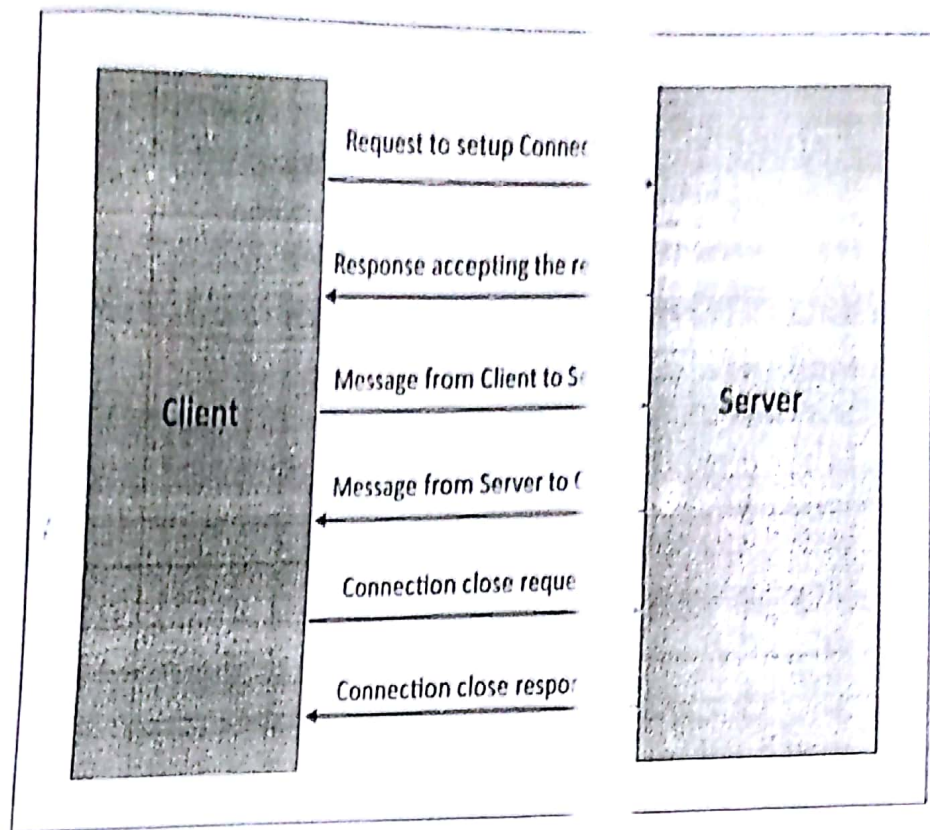


Figure 1.10: Exclusive Pair communication model

that uses a persistent connection between the client and server. Once the connection is setup it remains open until the client sends a request to close the connection. Client and server can send messages to each other after connection setup. Exclusive pair is a stateful communication model and the server is aware of all the open connections. Figure 1.10 shows the client-server interactions in the exclusive pair model.

and server. Once the connection is setup it remains open until the client sends a request to close the connection. Client and server can send messages to each other after connection setup. Exclusive pair is a stateful communication model and the server is aware of all the open connections. Figure 1.10 shows the client-server interactions in the exclusive pair model.

1.3.3 IoT Communication APIs

In the previous section you learned about various communication models. In this section you will learn about two specific communication APIs which are used in the examples in this book.

communication models. In this section you will learn about two specific communication APIs which are used in the examples in this book.

REST-based Communication APIs

Representational State Transfer (REST) [88] is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. REST APIs follow the request-response communication model described in previous section. REST architectural constraints apply to the components, connectors, and data elements within a distributed hypermedia system. The REST architectural constraints are as follows:

architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. REST APIs follow the request-response communication model described in previous section. REST architectural constraints apply to the components, connectors, and data elements within a distributed hypermedia system. The REST architectural constraints are as follows:

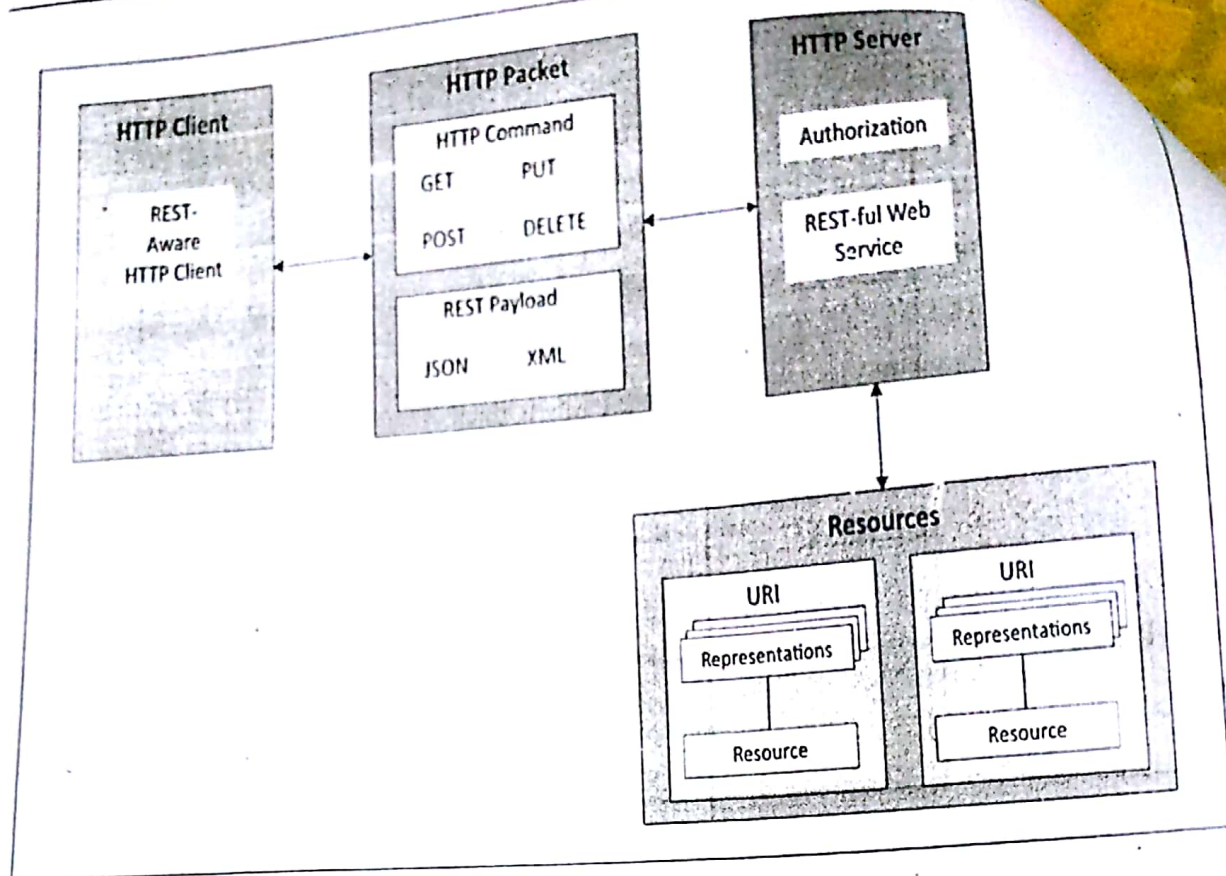


Figure 1.11: Communication with REST APIs

- **Client-Server:** The principle behind the client-server constraint is the separation of concerns. For example, clients should not be concerned with the storage of data which is a concern of the server. Similarly, the server should not be concerned about the user interface, which is a concern of the client. Separation allows client and server to be independently developed and updated.
- **Stateless:** Each request from client to server must contain all the information necessary to understand the request, and cannot take advantage of any stored context on the server. The session state is kept entirely on the client.
- **Cache-able:** Cache constraint requires that the data within a response to a request be implicitly or explicitly labeled as cache-able or non-cache-able. If a response is cache-able, then a client cache is given the right to reuse that response data for later, equivalent requests. Caching can partially or completely eliminate some interactions and improve efficiency and scalability.
- **Layered System:** Layered system constraint, constrains the behavior of components such that each component cannot see beyond the immediate layer with which they are interacting. For example, a client cannot tell whether it is connected directly to the end server, or to an intermediary along the way. System scalability can be improved

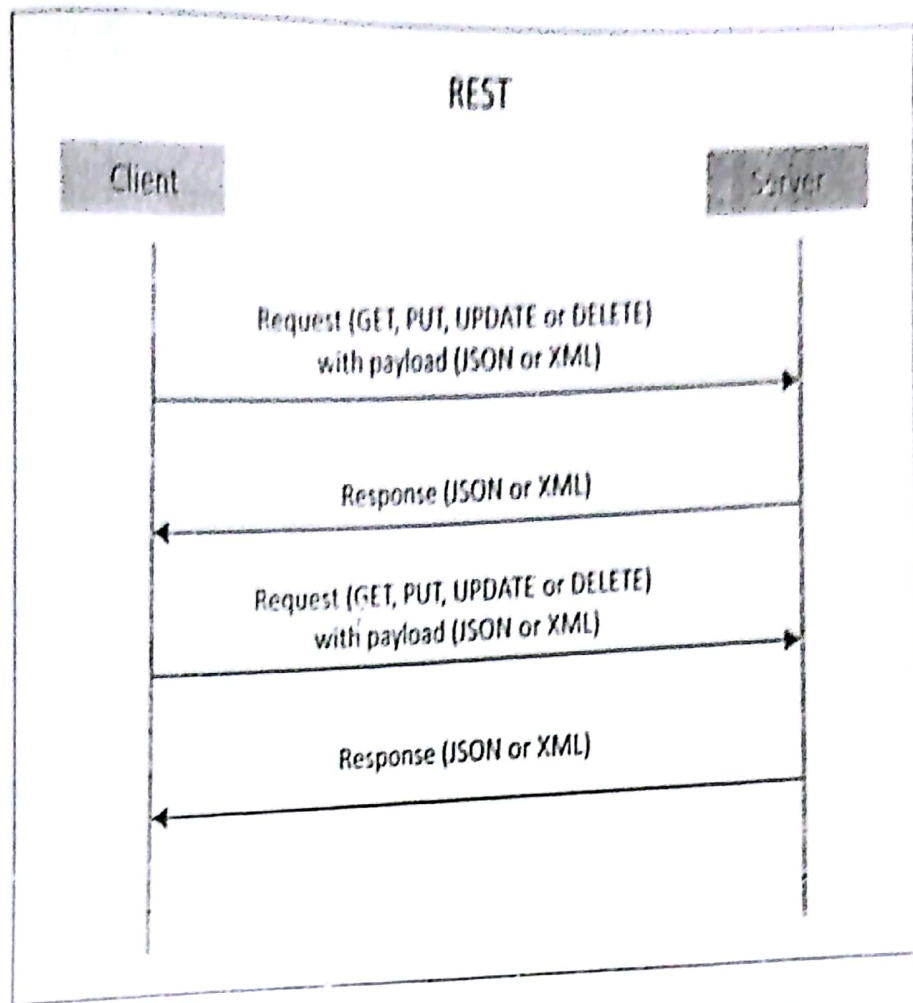


Figure 1.12: Request-response model used by REST

- by allowing intermediaries to respond to requests instead of the end server, without the client having to do anything different.
- **Uniform Interface:** Uniform Interface constraint requires that the method of communication between a client and a server must be uniform. Resources are identified in the requests (by URIs in web based systems) and are themselves separate from the representations of the resources that are returned to the client. When a client holds a representation of a resource it has all the information required to update or delete the resource (provided the client has required permissions). Each message includes enough information to describe how to process the message.
 - **Code on demand:** Servers can provide executable code or scripts for clients to execute in their context. This constraint is the only one that is optional.

A RESTful web service is a "web API" implemented using HTTP and REST principles. Figure 1.11 shows the communication between client and server using REST APIs. Figure 1.12 shows the interactions in the request-response model used by REST. RESTful web service is a collection of resources which are represented by URIs. RESTful web API has a base URI (e.g. <http://example.com/api/tasks/>). The clients send requests to these URIs using the

HTTP Method	Resource Type	Action	Example
GET	Collection URI	List all the resources in a collection	http://example.com/api/tasks/ (list all tasks)
GET	Element URI	Get information about a resource	http://example.com/api/tasks/1/ (get information on task-1)
POST	Collection URI	Create a new resource	http://example.com/api/tasks/ (create a new task from data provided in the request)
POST	Element URI	Generally not used	
PUT	Collection URI	Replace the entire collection with another collection	http://example.com/api/tasks/ (replace entire collection with data provided in the request)
PUT	Element URI	Update a resource	http://example.com/api/tasks/1/ (update task-1 with data provided in the request)
DELETE	Collection URI	Delete the entire collection	http://example.com/api/tasks/ (delete all tasks)
DELETE	Element URI	Delete a resource	http://example.com/api/tasks/1/ (delete task-1)

Table 1.1: HTTP request methods and actions

methods defined by the HTTP protocol (e.g., GET, PUT, POST, or DELETE), as shown in Table 1.1. A RESTful web service can support various Internet media types (JSON being the most popular media type for RESTful web services). IP for Smart Objects Alliance (IPSO Alliance) has published an Application Framework that defines a RESTful design for use in IP smart object systems [18].

WebSocket-based Communication APIs

WebSocket APIs allow bi-directional, full duplex communication between clients and servers. WebSocket APIs follow the exclusive pair communication model described in previous section and as shown in Figure 1.13. Unlike request-response APIs such as REST, the

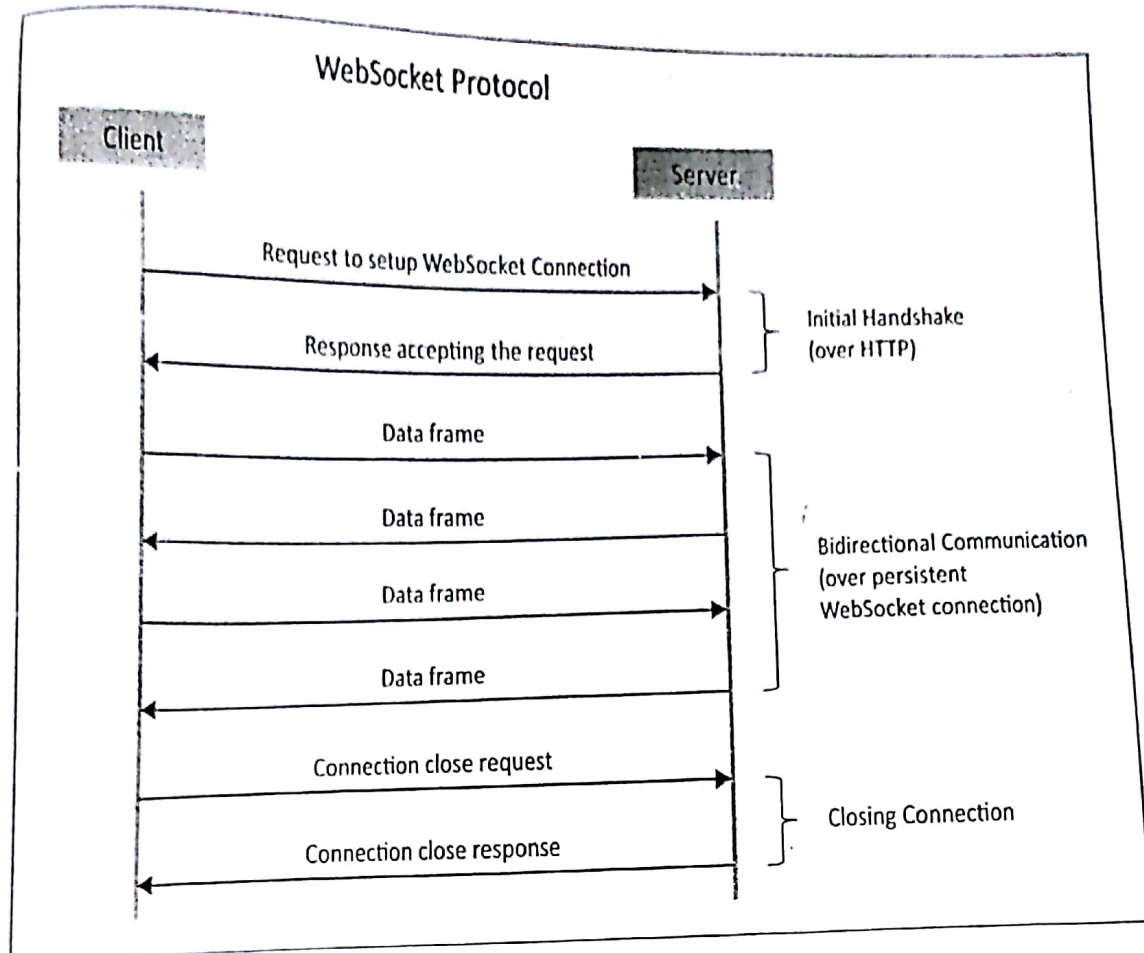


Figure 1.13: Exclusive pair model used by WebSocket APIs

WebSocket APIs allow full duplex communication and do not require a new connection to be setup for each message to be sent. WebSocket communication begins with a connection setup request sent by the client to the server. This request (called a WebSocket handshake) is sent over HTTP and the server interprets it as an upgrade request. If the server supports WebSocket protocol, the server responds to the WebSocket handshake response. After the connection is setup, the client and server can send data/messages to each other in full-duplex mode. WebSocket APIs reduce the network traffic and latency as there is no overhead for connection setup and termination requests for each message. WebSocket is suitable for IoT applications that have low latency or high throughput requirements.

1.4 IoT Enabling Technologies

IoT is enabled by several technologies including wireless sensor networks, cloud computing, big data analytics, embedded systems, security protocols and architectures, communication protocols, web services, mobile Internet, and semantic search engines. This section provides an overview of some of these technologies which play a key-role in IoT.

1.4.1 Wireless Sensor Networks

A Wireless Sensor Network (WSN) comprises of distributed devices with sensors which are used to monitor the environmental and physical conditions. A WSN consists of a number of end-nodes and routers and a coordinator. End nodes have several sensors attached to them. End nodes can also act as routers. Routers are responsible for routing the data packets from end-nodes to the coordinator. The coordinator collects the data from all the nodes. Coordinator also acts as a gateway that connects the WSN to the Internet. Some examples of WSNs used in IoT systems are described as follows:

- Weather monitoring systems use WSNs in which the nodes collect temperature, humidity and other data, which is aggregated and analyzed.
- Indoor air quality monitoring systems use WSNs to collect data on the indoor air quality and concentration of various gases.
- Soil moisture monitoring systems use WSNs to monitor soil moisture at various locations.
- Surveillance systems use WSNs for collecting surveillance data (such as motion detection data)
- Smart grids use WSNs for monitoring the grid at various points.
- Structural health monitoring systems use WSNs to monitor the health of structures (buildings, bridges) by collecting vibration data from sensor nodes deployed at various points in the structure.

WSNs are enabled by wireless communication protocols such as IEEE 802.15.4. ZigBee is one of the most popular wireless technologies used by WSNs. ZigBee specifications are based on IEEE 802.15.4. ZigBee operates at 2.4 GHz frequency and offers data rates upto 250 KB/s and range from 10 to 100 meters depending on the power output and environmental conditions. The power of WSNs lies in their ability to deploy large number of low-cost and low-power sensing nodes for continuous monitoring of environmental and physical conditions. WSNs are self-organizing networks. Since WSNs have large number of nodes, manual configuration for each node is not possible. The self-organizing capability of WSN makes the network robust. In the event of failure of some nodes or addition of new nodes to the network, the network can reconfigure itself.

1.4.2 Cloud Computing

Cloud computing is a transformative computing paradigm that involves delivering applications and services over the Internet. Cloud computing involves provisioning of computing, networking and storage resources on demand and providing these resources as metered services to the users, in a "pay as you go" model. Cloud computing resources can be provisioned on-demand by the users, without requiring interactions with the cloud service

provider. The process of provisioning resources is automated. Cloud computing resources can be accessed over the network using standard access mechanisms that provide platform-independent access through the use of heterogeneous client platforms such as workstations, laptops, tablets and smart-phones. The computing and storage resources provided by cloud service providers are pooled to serve multiple users using multi-tenancy. Multi-tenant aspects of the cloud allow multiple users to be served by the same physical hardware. Users are assigned virtual resources that run on top of the physical resources.

Cloud computing services are offered to users in different forms (see the authors' companion book on Cloud Computing, for instance):

- **Infrastructure-as-a-Service (IaaS)** : IaaS provides the users the ability to provision computing and storage resources. These resources are provided to the users as virtual machine instances and virtual storage. Users can start, stop, configure and manage the virtual machine instances and virtual storage. Users can deploy operating systems and applications of their choice on the virtual resources provisioned in the cloud. The cloud service provider manages the underlying infrastructure. Virtual resources provisioned by the users are billed based on a pay-per-use paradigm.
- **Platform-as-a-Service (PaaS)** : PaaS provides the users the ability to develop and deploy application in the cloud using the development tools, application programming interfaces (APIs), software libraries and services provided by the cloud service provider. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems and storage. The users, themselves, are responsible for developing, deploying, configuring and managing applications on the cloud infrastructure.
- **Software-as-a-Service (SaaS)** : SaaS provides the users a complete software application or the user interface to the application itself. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems, storage and application software, and the user is unaware of the underlying architecture of the cloud. Applications are provided to the user through a thin client interface (e.g., a browser). SaaS applications are platform independent and can be accessed from various client devices such as workstations, laptop, tablets and smart-phones, running different operating systems. Since the cloud service provider manages both the application and data, the users are able to access the applications from anywhere.

1.4.3 Big Data Analytics

Big data is defined as collections of data sets whose volume, velocity (in terms of its temporal variation), or variety, is so large that it is difficult to store, manage, process and analyze the data using traditional databases and data processing tools. Big data analytics involves

1.4 IoT Enabling Technologies

several steps starting from data cleansing, data munging (or wrangling), data processing, and data visualization. Some examples of big data generated by IoT systems are described as follows.

- Sensor data generated by IoT systems such as weather monitoring stations.
- Machine sensor data collected from sensors embedded in industrial and energy systems for monitoring their health and detecting failures.
- Health and fitness data generated by IoT devices such as wearable fitness bands.
- Data generated by IoT systems for location and tracking of vehicles.
- Data generated by retail inventory monitoring systems.

The underlying characteristics of big data include:

- **Volume:** Though there is no fixed threshold for the volume of data to be considered as big data, however, typically, the term big data is used for massive scale data that is difficult to store, manage and process using traditional databases and data processing architectures. The volumes of data generated by modern IT, industrial, and health-care systems, for example, is growing exponentially driven by the lowering costs of data storage and processing architectures and the need to extract valuable insights from the data to improve business processes, efficiency and service to consumers.
- **Velocity:** Velocity is another important characteristic of big data and the primary reason for exponential growth of data. Velocity of data refers to how fast the data is generated and how frequently it varies. Modern IT, industrial and other systems are generating data at increasingly higher speeds.
- **Variety:** Variety refers to the forms of the data. Big data comes in different forms such as structured or unstructured data, including text data, image, audio, video and sensor data.

1.4.4 Communication Protocols

Communication protocols form the backbone of IoT systems and enable network connectivity and coupling to applications. Communication protocols allow devices to exchange data over the network. In section 1.2.2 you learned about various link, network, transport and application layer protocols. These protocols define the data exchange formats, data encoding, addressing schemes for devices and routing of packets from source to destination. Other functions of the protocols include sequence control (that helps in ordering packets determining lost packets), flow control (that helps in controlling the rate at which the sender is sending the data so that the receiver or the network is not overwhelmed) and retransmission of lost packets.

1.4.5 Embedded Systems

An Embedded System is a computer system that has computer hardware and software embedded to perform specific tasks. In contrast to general purpose computers or personal computers (PCs) which can perform various types of tasks, embedded systems are designed to perform a specific set of tasks. Key components of an embedded system include, microprocessor or microcontroller, memory (RAM, ROM, cache), networking units (Ethernet, WiFi adapters), input/output units (display, keyboard, etc.) and storage (such as flash memory). Some embedded systems have specialized processors such as digital signal processors (DSPs), graphics processors and application specific processors. Embedded systems run embedded operating systems such as real-time operating systems (RTOS). Embedded systems range from low-cost miniaturized devices such as digital watches to devices such as digital cameras, point of sale terminals, vending machines, appliances (such as washing machines), etc. In the next chapter we describe how such devices form an integral part of IoT systems.

1.5 IoT Levels & Deployment Templates

In this section we define various levels of IoT systems with increasing complexity. An IoT system comprises of the following components:

- **Device:** An IoT device allows identification, remote sensing, actuating and remote monitoring capabilities. You learned about various examples of IoT devices in section 1.2.1.
- **Resource:** Resources are software components on the IoT device for accessing, processing, and storing sensor information, or controlling actuators connected to the device. Resources also include the software components that enable network access for the device.
- **Controller Service:** Controller service is a native service that runs on the device and interacts with the web services. Controller service sends data from the device to the web service and receives commands from the application (via web services) for controlling the device.
- **Database:** Database can be either local or in the cloud and stores the data generated by the IoT device.
- **Web Service:** Web services serve as a link between the IoT device, application, database and analysis components. Web service can be either implemented using HTTP and REST principles (REST service) or using WebSocket protocol (WebSocket service). A comparison of REST and WebSocket is provided below:

1.5 IoT Levels & Deployment Templates

- **Stateless/Stateful:** REST services are stateless in nature. Each request contains all the information needed to process it. Requests are independent of each other. WebSocket on the other hand is stateful in nature where the server maintains the state and is aware of all the open connections.
- **Uni-directional/Bi-directional:** REST services operate over HTTP and are uni-directional. Request is always sent by a client and the server responds to the requests. On the other hand, WebSocket is a bi-directional protocol and allows both client and server to send messages to each other.
- **Request-Response/Full Duplex:** REST services follow a request-response communication model where the client sends requests and the server responds to the requests. WebSocket on the other hand allow full-duplex communication between the client and server, i.e., both client and server can send messages to each other independently.
- **TCP Connections:** For REST services, each HTTP request involves setting up a new TCP connection. WebSocket on the other hand involves a single TCP connection over which the client and server communicate in a full-duplex mode.
- **Header Overhead:** REST services operate over HTTP, and each request is independent of others. Thus each request carries HTTP headers which is an overhead. Due the overhead of HTTP headers, REST is not suitable for real-time applications. WebSocket on the other hand does not involve overhead of headers. After the initial handshake (that happens over HTTP), the client and server exchange messages with minimal frame information. Thus WebSocket is suitable for real-time applications.
- **Scalability:** Scalability is easier in the case of REST services as requests are independent and no state information needs to be maintained by the server. Thus both horizontal (scaling-out) and vertical scaling (scaling-up) solutions are possible for REST services. For WebSockets, horizontal scaling can be cumbersome due to the stateful nature of the communication. Since the server maintains the state of a connection, vertical scaling is easier for WebSockets than horizontal scaling.
- **Analysis Component:** The Analysis Component is responsible for analyzing the IoT data and generate results in a form which are easy for the user to understand. Analysis of IoT data can be performed either locally or in the cloud. Analyzed results are stored in the local or cloud databases.
- **Application:** IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view the processed data.

1.5.1 IoT Level-1

A level-1 IoT system has a single node/device that performs sensing and/or actuation, stores data, performs analysis and hosts the application as shown in Figure 1.14. Level-1 IoT systems are suitable for modeling low-cost and low-complexity solutions where the data involved is not big and the analysis requirements are not computationally intensive.

Let us now consider an example of a level-1 IoT system for home automation. The system consists of a single node that allows controlling the lights and appliances in a home remotely. The device used in this system interfaces with the lights and appliances using electronic relay switches. The status information of each light or appliance is maintained in a local database. REST services deployed locally allow retrieving and updating the state of each light or appliance in the status database. The controller service continuously monitors the state of each light or appliance (by retrieving state from the database) and triggers the relay switches accordingly. The application which is deployed locally has a user interface for controlling the lights or appliances. Since the device is connected to the Internet, the application can be accessed remotely as well.

1.5.2 IoT Level-2

A level-2 IoT system has a single node that performs sensing and/or actuation and local analysis as shown in Figure 1.15. Data is stored in the cloud and application is usually cloud-based. Level-2 IoT systems are suitable for solutions where the data involved is big, however, the primary analysis requirement is not computationally intensive and can be done locally itself.

Let us consider an example of a level-2 IoT system for smart irrigation. The system consists of a single node that monitors the soil moisture level and controls the irrigation system. The device used in this system collects soil moisture data from sensors. The controller service continuously monitors the moisture levels. If the moisture level drops below a threshold, the irrigation system is turned on. For controlling the irrigation system actuators such as solenoid valves can be used. The controller also sends the moisture data to the computing cloud. A cloud-based REST web service is used for storing and retrieving moisture data which is stored in the cloud database. A cloud-based application is used for visualizing the moisture levels over a period of time, which can help in making decisions about irrigation schedules.

1.5.3 IoT Level-3

A level-3 IoT system has a single node. Data is stored and analyzed in the cloud and application is cloud-based as shown in Figure 1.16. Level-3 IoT systems are suitable for

1.5 IoT Levels & Deployment Templates

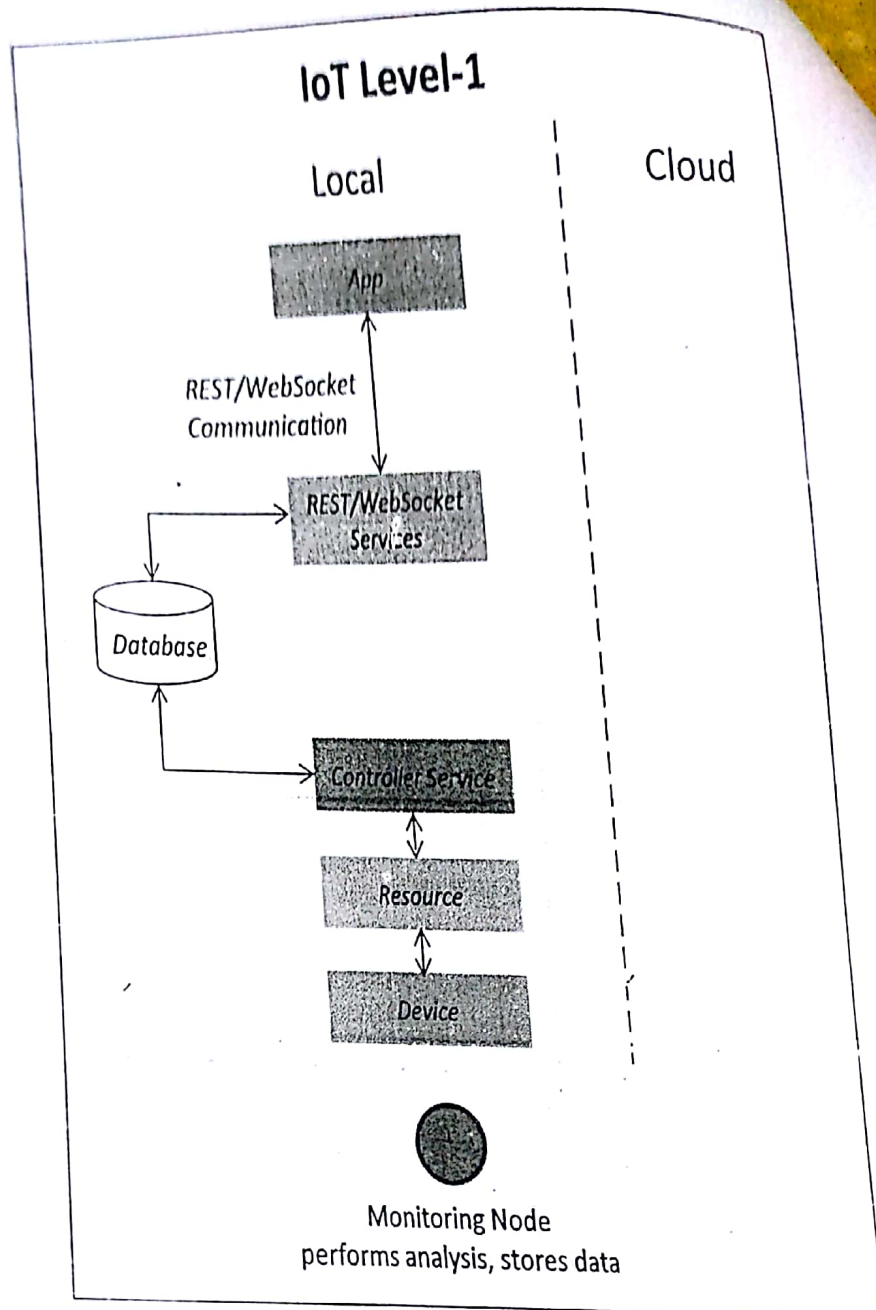


Figure 1.14: IoT Level-1

solutions where the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-2 IoT system for tracking package handling. The system consists of a single node (for a package) that monitors the vibration levels for a package being shipped. The device in this system uses accelerometer and gyroscope sensors for monitoring vibration levels. The controller service sends the sensor data to the cloud in real-time using a WebSocket service. The data is stored in the cloud and also visualized using a cloud-based application. The analysis components in the cloud can trigger alerts if

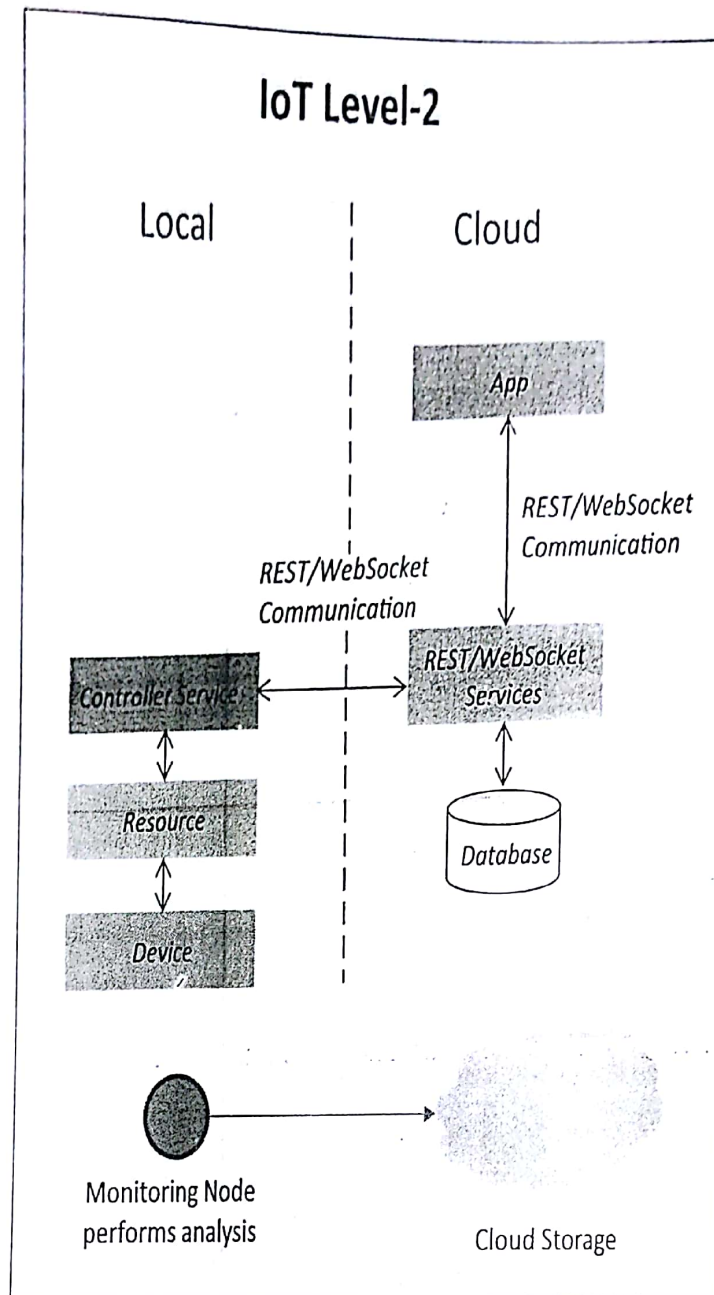


Figure 1.15: IoT Level-2

the vibration levels become greater than a threshold. The benefit of using WebSocket service instead of REST service in this example is that, the sensor data can be sent in real time to the cloud. Moreover, cloud based applications can subscribe to the sensor data feeds for viewing the real-time data.

1.5.4 IoT Level-4

A level-4 IoT system has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud-based as shown in Figure 1.17. Level-4 contains local and

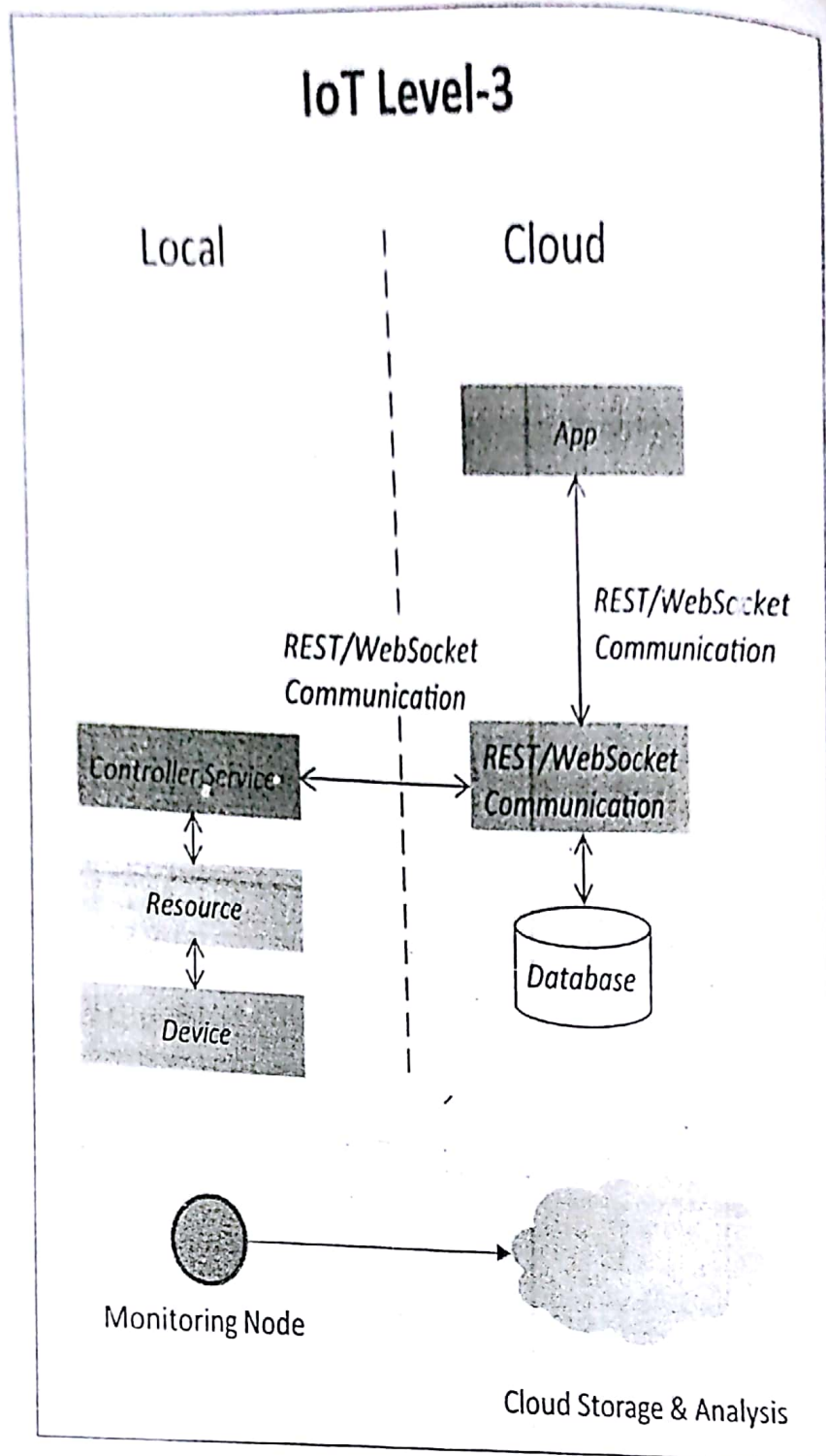


Figure 1.16: IoT Level-3

cloud-based observer nodes which can subscribe to and receive information collected in the cloud from IoT devices. Observer nodes can process information and use it for various applications, however, observer nodes do not perform any control functions. Level-4 IoT systems are suitable for solutions where multiple nodes are required, the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-4 IoT system for noise monitoring. The system consists of multiple nodes placed in different locations for monitoring noise levels in an area.

The nodes in this example are equipped with sound sensors. Nodes are independent of each other. Each node runs its own controller service that sends the data to the cloud. The data is stored in a cloud database. The analysis of data collected from a number of nodes is done in the cloud. A cloud-based application is used for visualizing the aggregated data.

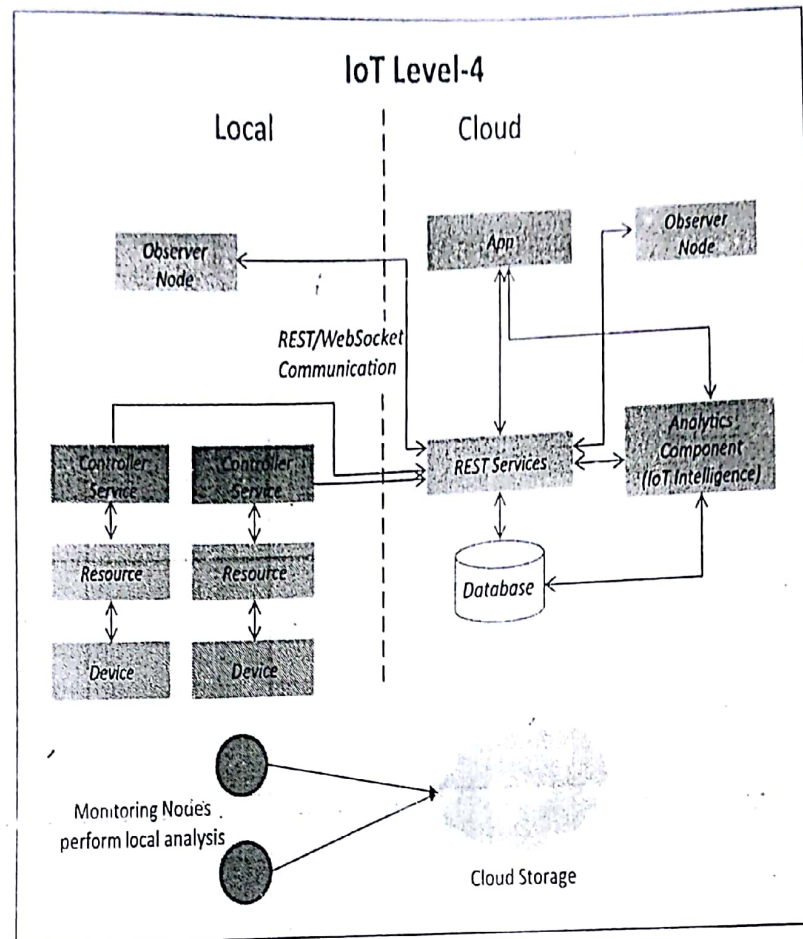


Figure 1.17: IoT Level-4

1.5.5 IoT Level-5

A level-5 IoT system has multiple end nodes and one coordinator node as shown in Figure 1.18. The end nodes that perform sensing and/or actuation. Coordinator node collects data from the end nodes and sends to the cloud. Data is stored and analyzed in the cloud and application is cloud-based. Level-5 IoT systems are suitable for solutions based on wireless sensor networks, in which the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-5 IoT system for forest fire detection. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and carbon dioxide (CO_2) levels in a forest. The end nodes in this example are equipped with

1.5 IoT Levels & Deployment Templates

various sensors (such as temperature, humidity and CO_2). The coordinator node receives the data from the end nodes and acts as a gateway that provides Internet connectivity to the IoT system. The controller service on the coordinator device sends the collected data to the cloud. The data is stored in a cloud database. The analysis of data is done in the computing cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data.

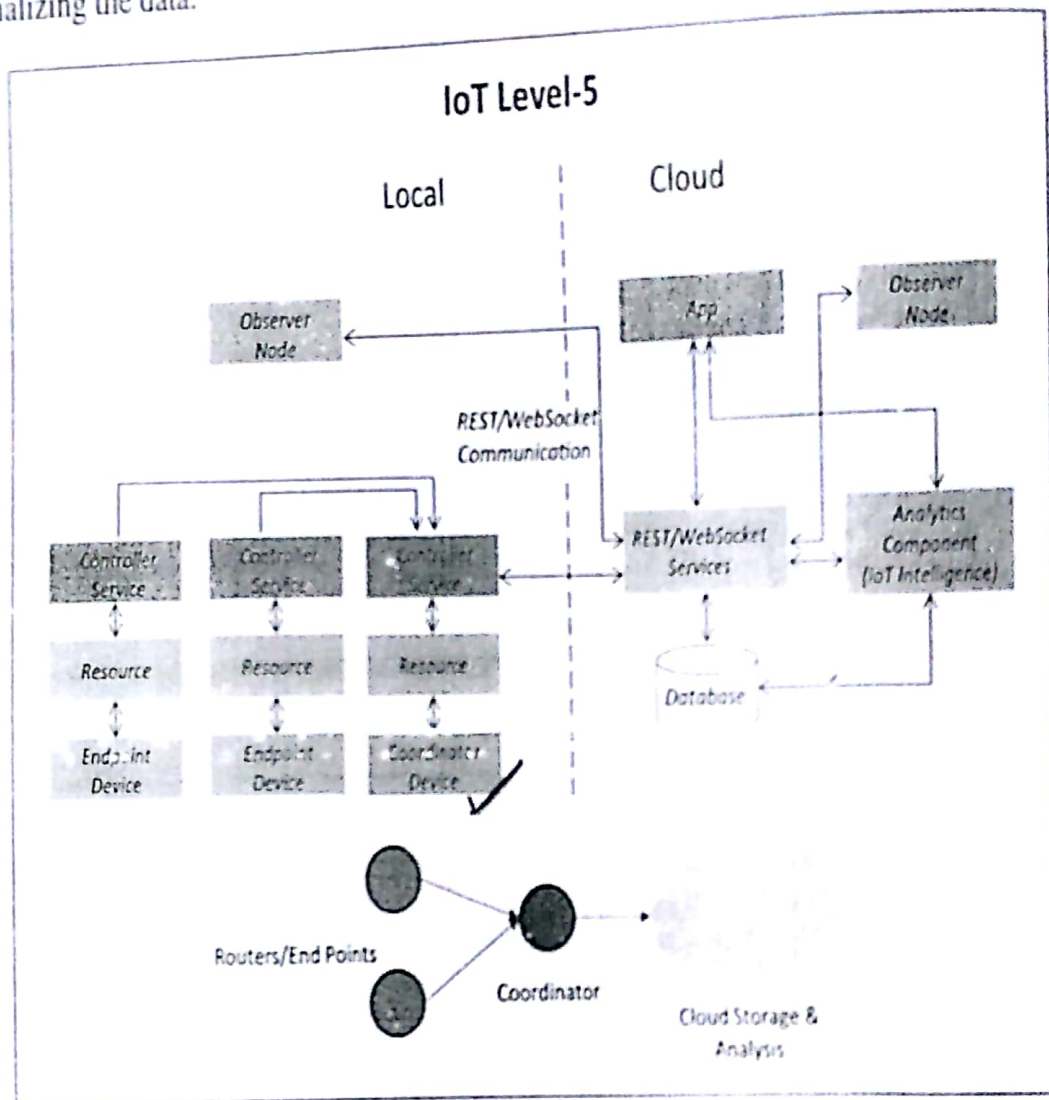


Figure 1.18: IoT Level-5

1.5.6 IoT Level-6

A level-6 IoT system has multiple independent end nodes that perform sensing and/or actuation and send data to the cloud. Data is stored in the cloud and application is cloud-based as shown in Figure 1.19. The analytics component analyzes the data and stores the results in the cloud database. The results are visualized with the cloud-based application. The centralized controller is aware of the status of all the end nodes and sends control commands

to the nodes.

Let us consider an example of a level-6 IoT system for weather monitoring. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and pressure in an area. The end nodes are equipped with various sensors (such as temperature, pressure and humidity). The end nodes send the data to the cloud in real-time using a WebSocket service. The data is stored in a cloud database. The analysis of data is done in the cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data.

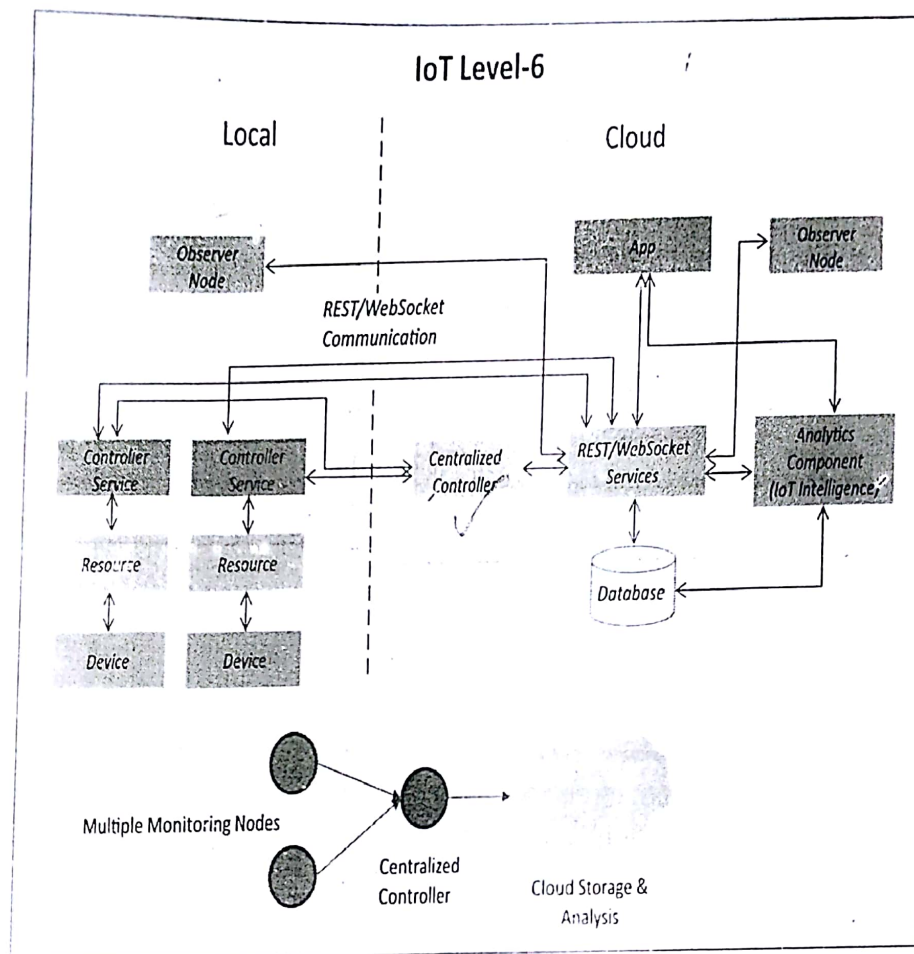


Figure 1.19: IoT Level-6

Summary

Internet of Things (IoT) refers to physical and virtual objects that have unique identities and are connected to the Internet. This allows the development of intelligent applications that make energy, logistics, industrial control, retail, agriculture and many other domains of

1.5 IoT Levels & Deployment Templates

human endeavour "smarter". IoT allows different types of devices, appliances, machines to communicate and exchange data. The applications of Internet of Things span a wide range of domains including (but not limited to) homes, cities, environment, energy systems, retail, logistics, industry, agriculture and health. Things in IoT refers to IoT devices which have unique identities and allow remote sensing, actuating and remote monitoring capabilities. Almost all IoT devices generate data in some form or the other which when processed by data analytics systems leads to useful information to guide further actions. You learned about IoT protocols for link, network, transport and application layers. Link layer protocols determine how the data is physically sent over the network to the destination network. The transport layer protocols provides end-to-end message transfer capability independent of the underlying network. Application layer protocols define how the applications interface with the lower layer protocols to ~~send the data over the network.~~ You learned about functional blocks of an IoT system including device communication, services, management, security and application blocks. You learned about IoT communication models such as request-response, publish-subscribe, push-pull and exclusive pair. You learned about REST-based and WebSocket-based communication APIs. REST is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. A RESTful web service is a web API implemented using HTTP and REST principles. WebSocket APIs allow bi-directional, full duplex communication between clients and servers. You learned about enabling technologies of IoT such as wireless sensor networks, cloud computing, big data analytics, communication protocols and embedded systems. Finally, you learned about IoT levels. A level-1 IoT system has a single node/device that performs sensing and/or actuation, stores data, performs analysis and hosts the application. A level-2 IoT system has a single node that performs sensing and/or actuation and local analysis. A level-3 IoT system has a single node. Data is stored and analyzed in the cloud and application is cloud-based. A level-4 IoT system has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud-based. A level-5 IoT system has multiple end nodes and one coordinator node. A level-6 IoT system has multiple independent end nodes that perform sensing and/or actuation and send data to the cloud.

Review Questions

1. Describe an example of an IoT system in which information and knowledge are inferred from data.
2. Why do IoT systems have to be self-adapting and self-configuring?

3. What is the role of things and Internet in IoT?
4. What is the function of communication functional block in an IoT system?
5. Describe an example of IoT service that uses publish-subscribe communication model.
6. Describe an example of IoT service that uses WebSocket-based communication.
7. What are the architectural constraints of REST?
8. What is the role of a coordinator in wireless sensor network?
9. What is the role of a controller service in an IoT system?

capabilities such as addressing, communication, and service discovery, among others, are delivered efficiently in both small and large scale. There is a need for enough name space to support increasing populations of devices and new applications. In particular, note that IPv6 is an ideal component (but not the only one) to be employed to support scalability, both for a given application as it reaches more users and for use for a wide class of applications spanning many fields (as described in Chapter 3).

4.2.4 Interoperability

Because of the plethora of applications, technology suppliers, and stakeholders, it is desirable to develop and/or re-use a core set of common standards. To the degree possible, existing standards may prove advantageous to a rapid and cost-effective deployment of the technology. Product and service interoperability is of interest.

4.2.5 Security and Privacy

Unfortunately, security is chronically an after-thought when it comes to protocol development: almost invariably a protocol spec will have many pages of data format and operation procedures and only a short paragraph or two on security considerations. When IoT relates to electric power distribution, goods distribution, transport and traffic management, e-health, and other key applications, as noted earlier, it is critical to maintain system-wide confidentiality, identity integrity, and trustworthiness.

4.2.6 Open Architecture

The goal is to support a wide range of applications using a common infrastructure, preferably based on a service-oriented architecture (SOA) over an open service platform, and utilizing overlay networks (these being logical networks defined on top of a physical infrastructure). In an SOA environment, objects expose their functionalities using a protocol such as SOAP or REST application programming interface (API). These devices may provide their functionality as a WS that can in turn be used by other entities (other devices or other business applications).

4.3 KEY IoT TECHNOLOGIES

There are a number of key supportive technologies that are needed for wide-scale deployment of IoT applications. This list is not exhaustive.

4.3.1 Device Intelligence

A key consideration relates to on-board intelligence. In order for the IoT to become a reality, the objects should be able to intelligently sense and interact with the environment, possibly store some passive or acquired data, and communicate with the world around them. Object-to-gateway device communication, or even direct object-to-object communication, is desirable. These intelligent capabilities are necessary to support the ubiquitous networking to provide seamlessly interconnection between

humans and objects. Some have called this mode of communication *Any Services, Any Time, Any Where, Any Devices, and Any Networks* (also known as "5-Any") (1). Pervasive computing (also known as ubiquitous computing) deals with the embedded ability to support logical processing as well with the ability to be in continuous range of a wireless gateway peer.

4.3.2 Communication Capabilities

As just noted, it is highly desirable for objects to support ubiquitous end-to-end communications; hence, another technological consideration relates to communication mechanisms. To achieve ubiquitous connectivity human-to-object and object-to-object communications, networking capabilities will need to be implemented in the objects ("things"). In particular, IP is considered to be key capability for IoT objects; furthermore, the entire TCP/IP Internet Suite is generally desirable. Self-configuring capabilities, especially how an IoT device can establish its connectivity automatically without human intervention, are also of interest. IPv6 auto-configuration and multihoming features are useful in this context, particularly the scope-based IPv6 addressing features.

While we have discussed objects that have sophisticated capabilities (IP support, IPv6 support, Web server capabilities, and so on) in the past few paragraphs, some applications, especially those using simple sensors and/or where there is a very large number of dispersed sensors and/or where there is limited remote energizing power, may have a need to support leaner protocols both at the network layer (e.g., route and/or topology management) and at the transport layer (e.g., using UDP). This may entail some extensions of existing networking protocols to achieve a level of simplicity and minimize power consumption. For constrained objects that do not have high levels of energizing power, memory, and/or computing, lightweight protocols that minimize energy consumption is a desiderata; however, one needs to keep in mind that these protocols may not have enough capabilities to support advanced applications. It should be noted that some existing applications may not even support the IP protocol (even IPv4) and the IP addressing scheme. Hence, there is a need to support heterogeneous (IP and non-IP) networking interfaces, at least in the short term. There may be a need for proxy gateways; such gateways would support multiple interfaces that have evolved from different heterogeneous networks. Interoperability among heterogeneous interfaces can facilitate commercial deployment.

4.3.3 Mobility Support

Yet another consideration relates to tracking and mobility support of mobile object (1). (Mobility-enabled architectures and protocols are required. Some objects move independently, while others will move as one of group. Therefore, according to the moving feature, different tracking methods are required. It is important to provide ubiquitous and seamless communication among objects while tracking the location of objects. Mobile IPv6 (MIPv6) offers several capabilities that can address this requirement.

4.3.4 Device Power

A key consideration relates to the powering of the "thing," especially for mobile devices or for devices that otherwise would not have intrinsic power. M2M/IoT applications are almost invariably constrained by the following factors: devices have ultra-low-power capabilities, devices must be of low cost, and devices generally must have small physical size and be light. Specifically, efficient communication mechanisms are needed. A number of devices operate with a small battery, while other devices use a self-energizing energy source, for example a small solar cell array. Yet other devices are passive (e.g., passive RFID) and, thus, need to derive energy indirectly from the environment, such as an intercepting electric/magnetic field. The power requirement is driven by the need to operate for extended periods of time from small batteries or from energy-scavenger mechanisms. In general, wireless technologies require significant amounts of power; hence, the need for low energy (LE) wireless technologies, as discussed in Chapter 6. Batteries are critical to all sorts of products including laptops, pads, smartphones, and IoT objects. The so-called "coin batteries," also known as "button batteries," are typical in many IoT applications.

In recent years, battery technology has seen a doubling in performance approximately every 10 (some say, 15) years. Unfortunately, battery technology does not follow Moore's Law which observes empirically that computer chips double in performance and drop their price 50% every 18 to 24 months.

Batteries convert chemical energy released in particular chemical reactions into electrical energy. Batteries have a positive and a negative electrode (the cathode and the anode), separated by an electrolyte. When the electrodes are connected to a closed circuit, a series of chemical reactions occurs such that at one end charged particles (ions) from the electrolyte flow to the anode, react, and free up electrons; at the other end, reactions at the cathode attract free electrons. Thus, electrons at the anode move to the cathode and the flow of electrons through the electric circuit creates an electric current—the electrolyte also prevents the electrons from taking the shortest direct path, instead forcing them through the attached circuit. In rechargeable batteries, the reactions are reversible, with the ions and electrons flowing back in the opposite direction during charging. Batteries can be classified into primary and secondary systems (8). Primary batteries are disposable batteries, that is, batteries that cannot be recharged, and their conversion of chemical energy into electrical energy is irreversible (the chemicals are consumed while the battery discharges). Secondary batteries can be recharged, and the electrode material is reconstituted using an electric charge, so that discharge process can be repeated a number of times during the lifecycle of the battery.

The most common primary systems are alkaline, lithium, and metal/air batteries. Among secondary batteries, lead acid, nickel/cadmium (NiCd), nickel/metal hybrid (NiMH), and lithium-ion (Liion)/lithium-polymer (Li-polymer) batteries dominate the market, but efforts are constantly being made to find new systems that can match or exceed the performance of existing systems, improve their safety, and reduce their cost.

Rechargeable Li-ion batteries have an anode comprising carbon (e.g., graphite), a metal oxide cathode, and an electrolyte containing lithium salt. It is relatively easy to peel ions from lithium metal. The widespread deployment of this battery technology is due to the fact that the resulting batteries are lightweight, have a high energy density, hold their charge better than other batteries, and they do not suffer from the "memory effect," where batteries hold less and less charge over time if they are not drained and then recharged completely. The technology became popular in the early 1990s, replacing the nickel cadmium predecessors. In recent years, manufacturers have improved battery performance by applying enhanced engineering, optimizing the structures, and/or adding new materials inside the battery to make them more efficient. While battery technology is evolving, Li-ion batteries will continue to be important for the foreseeable future.

Materials such as silicon and others are being studied as possible replacement of the graphite anodes in Li-ion batteries. Silicon is of interest because it is inexpensive, it is abundant, and, by weight, it can store 10 times more lithium ions than graphite; this implies that it could theoretically allow a 10-fold increase in performance. However, to be useful, researchers must overcome a problem: while graphite anodes hold their shape when they soak up lithium ions, silicon swells, causing silicon particles to become separated, quickly reducing the performance of the battery. There is work underway to address this challenge, for example, by developing rubbery conductive binders that stick to the silicon particles within the anode, stretching and shrinking as the battery is charged and discharged. Others are looking to develop Li-ion batteries with anodes containing silicon nanowires. We are also starting to see a research on totally different chemistries. One example is lithium-air batteries with anodes made of lightweight porous carbon. Oxygen from the air enters the porous carbon and reacts with the lithium ions in the electrolyte and electrons in the external circuit to form solid lithium oxide. Recharging causes the lithium compound to decompose, releasing the lithium ions and releasing the oxygen; calculations of the amount of energy involved in the chemical reactions involved suggest it could produce batteries that last three to five times as long as existing Li-ion batteries. Other researchers are also investigating lightweight lithium-sulfur packs, which have a life span of three times that of current Li-ion batteries. Research work is proceeding on these and other technologies (9). However, these advances represent only incremental improvements in performance. New materials may be needed to make a quantum leap forward. MIT's Materials Project has already identified four new materials with the potential to be used in batteries. For example, the use of magnesium metal anode could result into a three-fold energy density compared with the best Li-ion batteries; furthermore these magnesium-ion batteries hold the majority of their charge over 3000 charge cycles. The technology, however, is not ready for widespread commercialization.

Another approach is the fuel cell, which is a proven technology since they were used to power electronics of the Gemini and Apollo space missions. Fuel cells convert chemical energy into electricity by converting the chemical energy from a fuel (e.g., alcohol) into electricity through a chemical reaction with oxygen. Fuel cells have a high energy density: hydrogen contains nearly 150 times the energy of an equivalent weight of lithium. However, to be practical, they need to be small and have an easily

rechargeable reservoir for fuel. Microelectromechanical system (MEMS) technology is being investigated for this purpose. MEMSs are miniaturized mechanical devices that are already used in solar cells and flat-screen TVs. Currently, the technology is expensive because precious metals such as platinum and palladium are used; companies such as, but not limited to, NEC, Toshiba, and Apple are continuing substantive research in the field.

(Some evolving technologies use small solar panels embedded in the screen of a smartphone or object; other systems may use kinetic devices that translate movement of objects into an electric current. Solar cells are an example of an energy harvester, but they are for low efficiency when converting ambient light into useful electrical energy. A 3 cm² solar cell (dimensions similar to the common CR2032 coin cell) yields only 12 μ W.

There are a number of factors that must be considered in selecting the most suitable battery for a particular application; key considerations include (8):

- Operating voltage level
- Load current and profile
 - Duty cycle—continuous or intermittent
 - Service life
 - Physical requirement
 - Size
 - Shape
 - Weight
 - Environmental conditions
 - Temperature
 - Pressure
 - Humidity
 - Vibration
 - Shock
 - Pressure
 - Safety and reliability
 - Shelf life
 - Maintenance and replacement
 - Environmental impact and recycling capability
 - Cost

4.3.5 Sensor Technology

A sensor network is an infrastructure comprising sensing (measuring), computing, and communication elements that gives the administrator the ability to instrument, observe, and react to events and phenomena in a specified environment. The administrator typically is some civil, government, commercial, or industrial entity.

Network(ed) sensor systems support a plethora of applications, not the least being Homeland Security. Typical applications include, but are not limited to, data collection, monitoring, surveillance, and medical telemetry. Sensors facilitate the instrumenting and controlling of factories, offices, homes, vehicles, cities, and the ambiance, especially as commercial off-the-shelf technology becomes available. With sensor network technology, specifically, with embedded networked sensing, ships, aircrafts, and buildings can "self-detect" structural faults (e.g., fatigue-induced cracks). Places of public assembly can be instrumented to detect airborne agents such as toxins and to trace the source of the contamination, should any be present (this also can be done for ground/underground situations). Earthquake-oriented sensors in buildings can locate potential survivors and can help assess structural damage; tsunami-alerting sensors can certainly prove useful for nations with extensive coastlines. Sensors also find extensive applicability in battlefield for reconnaissance and surveillance. In addition to sensing, one is often also interested in control and activation (13).

There are four basic components in a sensor network: (i) an assembly of distributed or localized sensors; (ii) an interconnecting network (usually, but not always, wireless-based); (iii) a central point of information clustering; and (iv) a set of computing resources at the central point (or beyond) to handle data correlation, event-trending, querying, and data mining. Because the interconnecting network is generally wireless, these systems are known as wireless sensor networks (WSNs).

In this context, the sensing and computation nodes are considered part of the sensor network; in fact, some of the computing may be done in the network itself. Because of the potentially large quantity of data collected, algorithmic methods for data management play an important role in sensor networks. The computation and communication infrastructure associated with sensor networks is often specific to this environment and rooted in the device- and application-based nature of these networks. For example, unlike most other settings, in-network processing is desirable in sensor networks; furthermore, node power (and/or battery life) is a key design consideration.

Sensors, the things or objects in this discussion, are active devices that measure some variable of the natural or man-made environment (e.g., a building, an assembly line, an industrial assemblage). Sensors in a WSN have a variety of purposes, functions, and capabilities. The radar networks used in air traffic control, the national electrical power grid, and the nation-wide weather stations deployed over a regular topographic mesh are all examples of early-deployment sensor networks. All of these systems, however, use specialized computers and communication protocols and are very expensive. Less expensive WSNs are now being planned for novel applications in physical security, healthcare, and commerce. The technology for sensing and control includes electric and magnetic field sensors; radio-wave frequency sensors; optical, electro-optic, and infrared sensors; radars; lasers; location/navigation sensors; seismic and pressure-wave sensors; environmental parameter sensors (e.g., wind, humidity, heat, and so on); and biochemical Homeland Security-oriented sensors.

Sensors can be described as "smart" inexpensive devices equipped with multiple on-board sensing elements: they are low cost, low power, untethered multifunctional nodes that are logically homed to a central sink node. Sensors are typically internetworked via a series of multihop short-distance low power wireless links

(particularly within a defined "sensor field"); they typically utilize the Internet or some other network for long-haul delivery of information to a point (or points) of final data aggregation and analysis. In general, within the "sensor field," WSNs employ contention-oriented random access channel sharing/transmission techniques that are now incorporated in the IEEE 802 family of standards; indeed, these techniques were developed in the late 1960s and 1970s expressly for wireless (not cabled) environments, and for large sets of dispersed nodes with limited channel-management intelligence. However, other channel management techniques are also available. Sensors are typically deployed in a high density manner and in large quantities: a WSN consists of densely distributed nodes that support sensing, signal processing, embedded computing, and connectivity; sensors are logically linked by self-organizing means (sensors that are deployed in short-hop point-to-point master-slave pair arrangements are also of interest). Wireless sensors typically transmit information to collecting (monitoring) stations that aggregate some or all of the information. WSNs have unique characteristics, such as, but not limited to, power constraints/limited battery life for the wireless sensors, redundant data, low duty cycle, and many-to-one flows. Consequently, new design methodologies are needed across a set of disciplines, including, but not limited to, information transport, network and operational management, confidentiality, integrity, availability, and in-network/local processing. In some cases, it is challenging to collect (extract) data from wireless nodes (WNs) because connectivity to/from the WNs may be intermittent due to a low battery status (e.g., if these are dependent on sun light to recharge), or other wireless sensor malfunction. Furthermore, a lightweight protocol stack is desired. Often, a very large number of client units (say 64K or more) need to be supported by the system and by the addressing apparatus.

Sensors span several orders of magnitude in physical size; they (or, at least some of their components) range from nanoscopic-scale devices to mesoscopic-scale devices at one end; and, from microscopic-scale devices to macroscopic-scale devices at the other end. Nanoscopic (also known as nanoscale) refers to objects or devices in the order of 1–100 nm in diameter; mesoscopic scale refers to objects between 100 and 10,000 nm in diameter; the microscopic scale ranges from 10 to 1000 microns; and the macroscopic scale is at the millimeter-to-meter range. At the low end of the scale, one finds, among others, biological sensors, small passive microsensors (such as "smart dust"), and "lab-on-a-chip" assemblies. At the other end of the scale, one finds platforms such as, but not limited to, identity tags, toll collection devices, controllable weather data collection sensors, bioterrorism sensors, radars, and undersea submarine traffic sensors based on sonars.² Some refer to the latest generation of sensors, especially the miniaturized ones that are directly embedded in some physical infrastructure, as "microsensors." Microsensors with on-board processing and wireless interfaces can be utilized to study and monitor a variety of phenomena and environments at close proximity.

Sensors may be passive and/or be self-powered; further along in the power-consumption chain, some sensors may require relatively low power from a battery

² While satellites can be used to support sensing, this book does not explicitly include them in the technical discussion.

or line feed. At the high end of the power-consumption chain, some sensors may require very high power feeds (e.g., for radars). Chemical-, physical-, acoustic-, and image-based sensors can be utilized to study ecosystems (e.g., in support of global parameters such as temperature, microorganism populations, and so on). Near-term commercial applications include, but are not limited to, industrial/building WSNs, appliance control (lighting and heating, ventilation, and air conditioning (HVAC)), automotive sensors and actuators, home automation and networking, automatic meter reading/load management (LM), consumer electronics/entertainment, and asset management. Commercial market segments include the following:

- Industrial monitoring and control
- Commercial building and control
- Process control
- Home automation
- Wireless automated meter reading (AMR)/ LM
- Metropolitan operations (traffic, automatic tolls, fire, and so on)
- Homeland Security applications: chemical, biological, radiological, and nuclear wireless sensors
- Military sensors
- Environmental (land, air, sea)/agricultural wireless sensors

Suppliers and products tend to cluster according to these categories.

Implementations of WSNs have to address a set of technical challenges; however, the move toward standardization will, in due course, minimize a number of these challenges by addressing the issues once and then resulting in off-the-shelf chipsets and components. One of the challenges of WSNs is the need for extended temporal operation of the sensing node in spite of a (typically) limited power supply (and/or battery life). In particular, the architecture of the radio, including the use of low power circuitry, must be properly selected. In practical terms, this implies low power consumption for transmission over low bandwidth channels and low power-consumption logic to pre-process and/or compress data. Energy-efficient wireless communications systems are being sought and are typical of WSNs. Low power consumption is a key factor in ensuring long operating horizons for non-power-fed systems (some systems can indeed be power-fed and/or relay on other power sources). Power efficiency in WSNs is generally accomplished in three ways:

- (i) Low duty cycle operation
- (ii) Local/in-network processing to reduce data volume (and, hence, transmission time)
- (iii) Multihop networking (this reduces the requirement for long-range transmission since signal path loss is an inverse power with range/distance (e.g., 4)—each node in the sensor network can act as a repeater, thereby reducing the link range coverage required, and, in turn, the transmission power)

Conventional wireless networks are generally designed with link ranges of the order of tens, hundreds, or thousands of miles. The reduced link range and the compressed data payload in WSNs result in characteristic link budgets that differ from conventional systems.

4.3.6 RFID Technology

RFIDs are electronic devices associated with objects ("things") that transmit their identity (usually a serial number) via radio links. The RFID space is large and well documented. Our discussion here is very limited by choice; the reader requiring more details is encouraged to seek out the literature on the topic.

RFID tags are devices that typically have a read-only chip that stores a unique number but has no processing capability. RFID tags have broad applications, including the rapid collection of data in commercial environments. For example, RFID and bar coding are nearly ubiquitous in the inventory process, providing both accuracy and speed of data collection. These technologies facilitate the global supply chain and impact all subsystems within that overall process, including material requirement planning (MRP), just in time (JIT), electronic data interchange (EDI), and electronic commerce (EC). RFIDs are also used in industrial environments, such as but not limited to dirty, wet, or harsh environments. The technology can also be used for identification of people or assets. Figure 4.1 depicts two illustrative examples of RFIDs. Figure 4.2 depicts the basic operation of an RFID system.

Contactless smart cards (SCs) are more sophisticated than RFID tags, being that they contain a microprocessor that enables (i) on-board computing, (ii) two-way

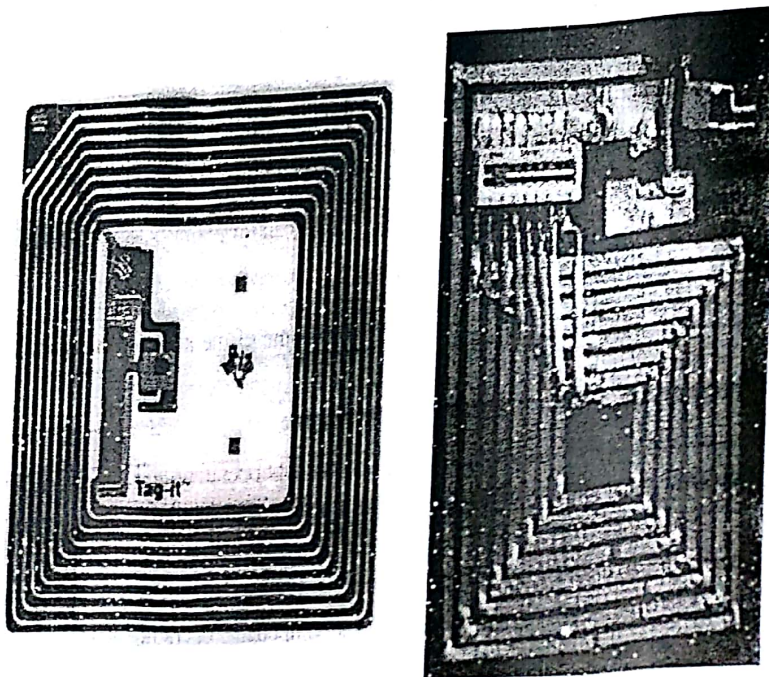


FIGURE 4.1 Illustrative examples of RFIDs.

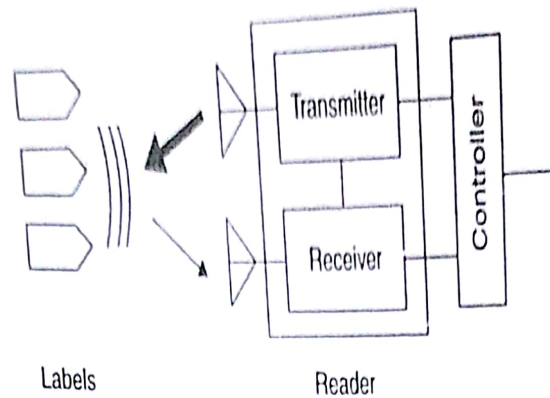


FIGURE 4.2 RFID reader operation.

communication including encryption, and (iii) storage of predefined and newly acquired information. Because of their more restricted capabilities, RFID tags are typically less expensive than SCs. When an RFID tag or contactless SC passes within a defined range, a reader generates electromagnetic waves; the tag's integrated antenna receives the signal and activates the chip in the tag/SC, and a wireless communications channel is set up between the reader and the tag enabling the transfer of pertinent data. Figure 4.3 provides a comparison between SCs and RFID tags.

RFID examples applicable to IoT include but are not limited to the following:

- Warehouse retailer automotive
- Grocery chain transportation
- Distribution center asset management
- Manufacturing
- Inventory management
- Warehousing and distribution
- Shop floor (production)
- Document tracking and asset management
- Industrial applications (e.g., time and attendance, shipping document tracking, receiving fixed assets)
- Retail applications

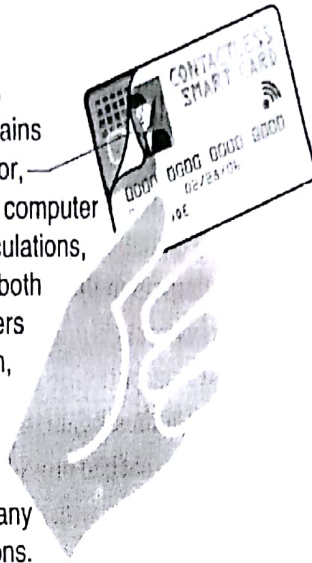
There are a number of standards for RFIDs. Some of the key ones include the following:

- The ISO 14443 standard describes components operating at 13.56 MHz frequency that embed a CPU; power consumption is about 10mW; data throughput is about 100 Kbps and the maximum working distance (from the reader) is around 10 cm.
- The ISO 15693 standard also describes components operating at 13.56 MHz frequency, but it enables working distances as high as 1 m, with a data throughput of a few Kbps.

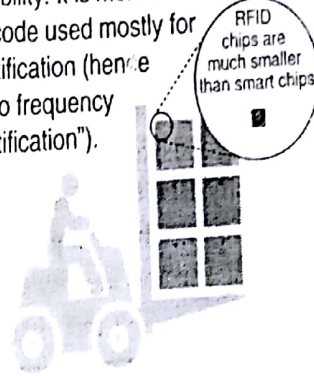
Overview: what happens in RF (radio frequency) communication

- 1 When a contactless smart card or an RFID tag passes within range, a reader sends out radio frequency electromagnetic waves.
- 2 The antenna, tuned to receive these waves, wakes up the chip in the smart card or tag.
- 3 A wireless communications channel is set up between the reader and the smart card or tag.

The contactless smart card contains a microprocessor, a small but real computer that makes calculations, communicates both ways, remembers new information, and actively uses these capabilities for security and many other applications.



RFID tags are devices that typically have a read-only chip that stores a unique number but has no processing capability. It is more like a radio-based bar code used mostly for identification (hence "radio frequency identification").



Characteristics of a contactless card

- Strong security capacities:
 - mutual authentication before providing access to information
 - access can be further protected via PIN or biometric
 - encryption to protect data on card during exchange
 - hardware and software protection to combat attacks or counterfeiting
- Hundreds of security features mean an individual's personal ID, financial details, payment transactions, transit fares or physical access privileges can be safely stored, managed, and exchanged
- Read and write memory capacity of 512 bytes and up, with very large memory storage possible
- Short-distance data exchange, typically two inches

Characteristics of an RFID tag

- Minimal security:
 - one-way authentication; card cannot protect itself
 - insufficient storage for biometrics
 - no on-chip calculations of new information
 - relies on static keys
- Single function; used to help machines identify objects to increase efficiency.
Example: inventory control
- Small memory (92 bytes); often read-only
- Larger distance data exchange, typically several yards

Because of their more restricted capabilities, RFID tags are generally cheaper.

FIGURE 4.3 Comparison between contactless SCs and RFID tags. Source: Gemalto (used with Permission).

- The ISO 18000 standard defines parameters for air interface communications associated with frequency such as 135 KHz, 13.56 MHz, 2.45 GHz, 5.8 GHz, 860–960 MHz, and 433 MHz. The ISO 18000–6 standard uses the 860–960 MHz range and is the basis for the Class-1 Generation-2 UHF RFID, introduced by the EPCglobal Consortium.

As a side note, EPCglobal Inc. was created as a joint venture between GSI (formerly EAN International) and GSI US (formerly the Uniform Code Council, Inc.)—the same organizations entrusted to drive adoption of the barcode—to develop standards and to create a “visible” global supply chain. EPCglobal is a neutral, not-for-profit standards organization consisting of manufacturers, technology solution providers, and retailers. Many industries participate in the EPCglobal standards development process such as aerospace, apparel, chemical, consumer electronics, consumer goods, healthcare and life sciences, and transportation and logistics.

Typically, EPC codes used for active RFIDs or IP addresses are transmitted in clear form; however, some new protocols are now emerging that can provide strong privacy for the IoT. The host identity protocol (HIP) is one example; with this protocol, active RFIDs do not expose their identity in clear text, but protect the identity value (e.g., an EPC) using cryptographic procedures (10).

Table 4.2 based on material from Reference 11 provides a very basic listing of RFID concepts. An RFID system is logically comprising several layers, as follows: the tag layer, the air interface (also called media interface) layer, and the reader layer; additionally there are network, middleware, and application aspects. Some of the key aspects of the basic layers are as follows:

- Tag (device) layer: Architecture and EPCglobal Gen2 tag finite state machine
- Media interface layer: Frequency bands, antennas, read range, modulation, encoding, data rates
- Reader layer: Architecture, antenna configurations, Gen2 sessions, Gen2

The following is a list of key specifications supporting basic RFID operations:

- EPCglobal™: *EPC™ Tag Data Standards*
- EPCglobal™ (2004): *FMCG RFID Physical Requirements Document*
- EPCglobal™ (2004): *Class-1 Generation-2 UHF RFID Implementation Reference*
- EPCglobal™ (2005): *Radio-Frequency Identity Protocols, Class-1 Generation-2 UHF RFID, Protocol for Communications at 860 MHz–960 MHz*
- European Telecommunications Standards Institute (ETSI), EN 302 208: *Electromagnetic Compatibility and Radio Spectrum Matters (ERM)—Radio-Frequency Identification Equipment Operating in the Band 865 MHz to 868 MHz with Power Levels up to 2 W, Part 1 – Technical Characteristics and Test Methods*